

POLITECNICO
MILANO 1863

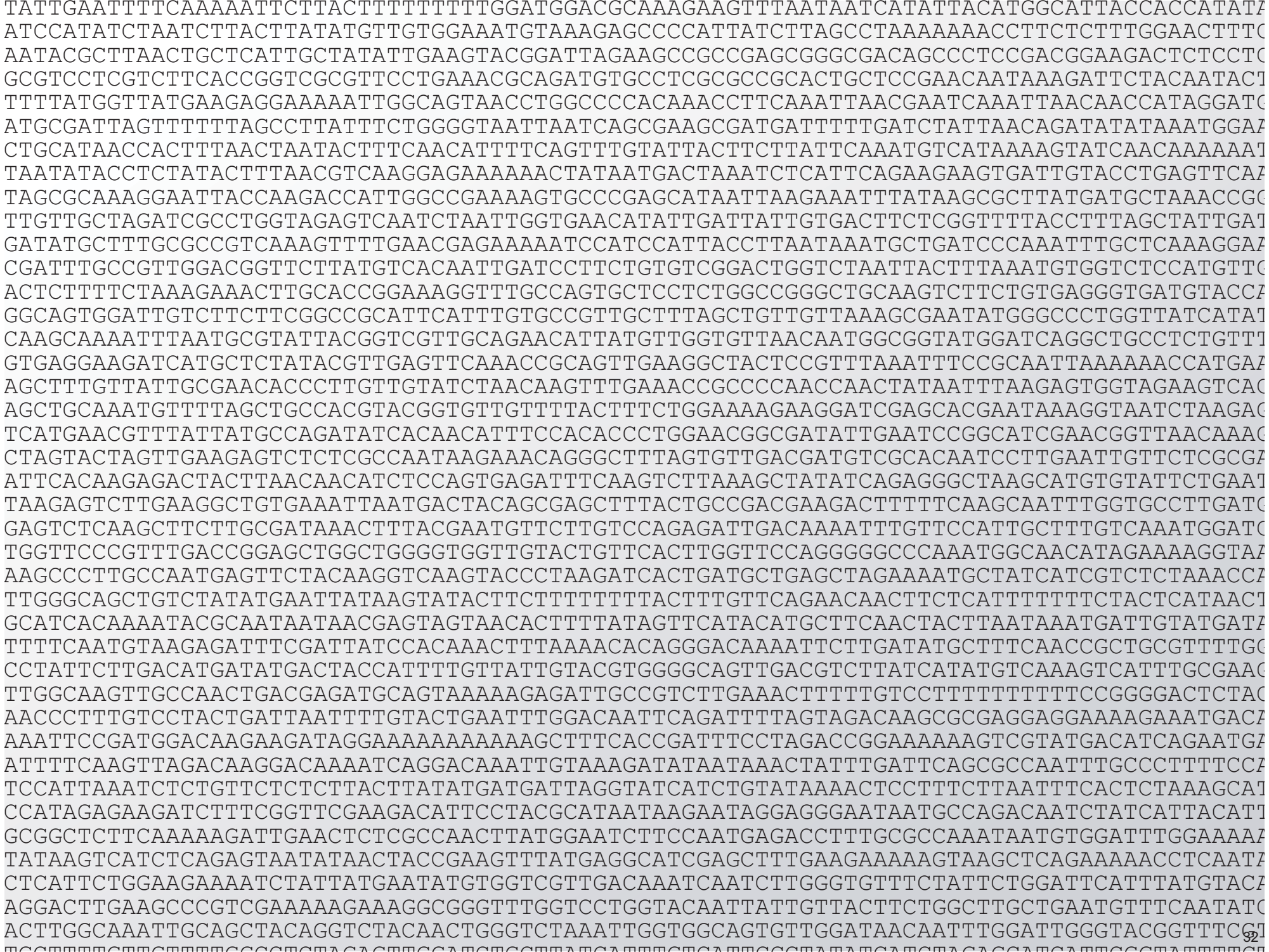
Bioinformatics Algorithms

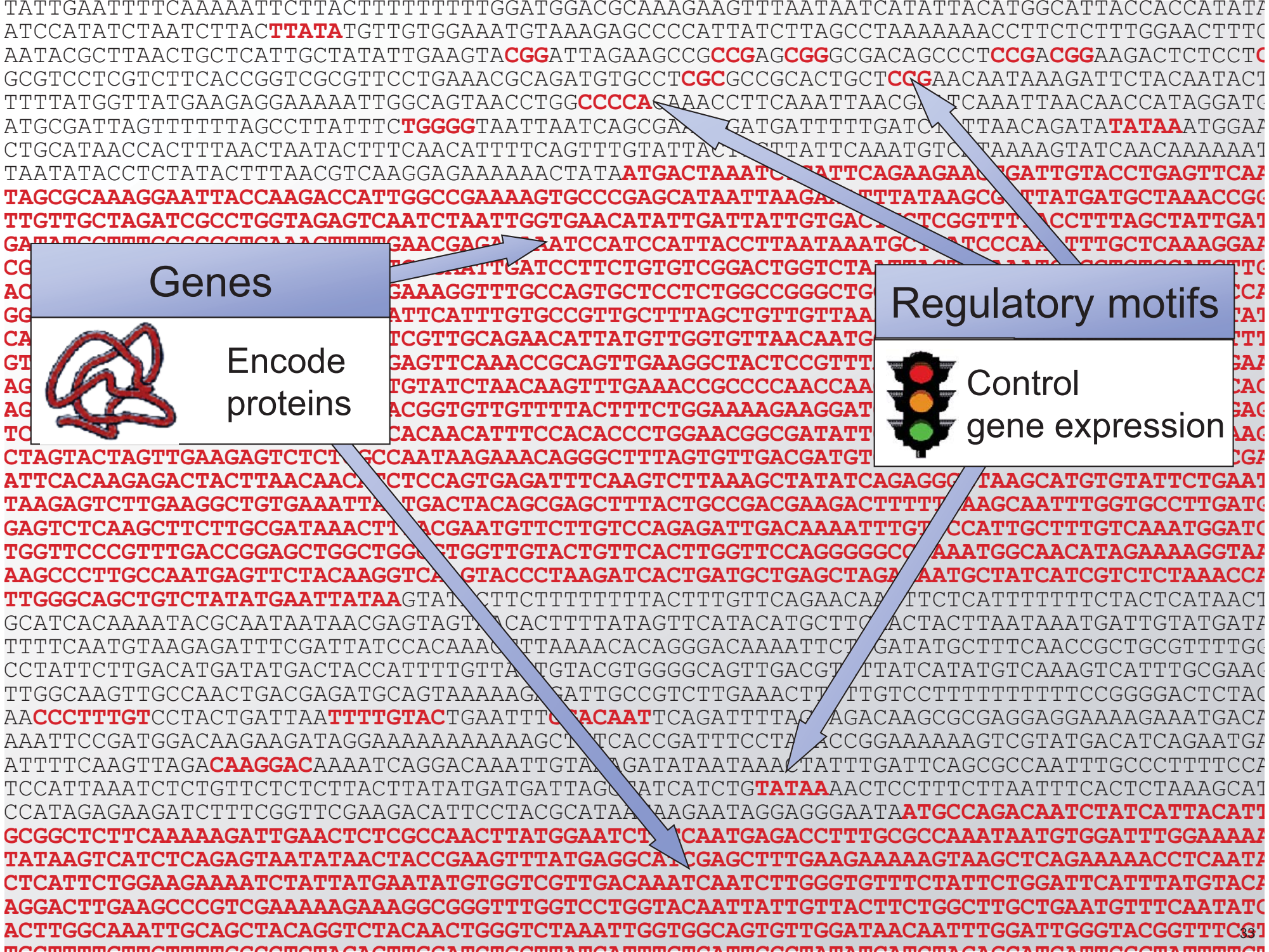
Hidden Markov Models for sequence modeling and pattern mining

Rosario M. Piro

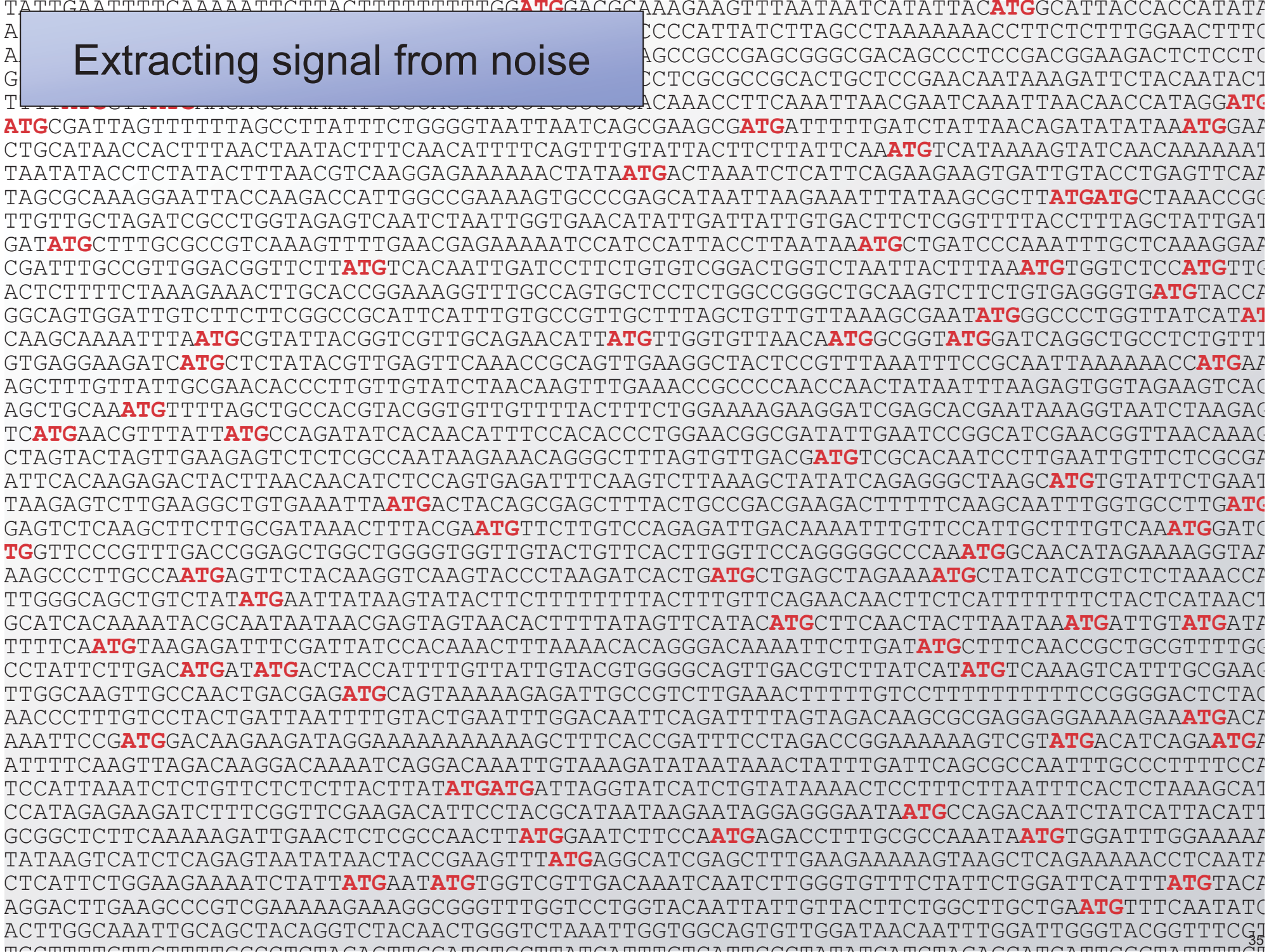
Dipartimento di Elettronica, Informazione e Bioingegneria (DEIB)
Politecnico di Milano

(most slides taken or adapted from: Manolis Kellis, MIT / Broad Institute)




Genes
Encode proteins
Regulatory motifs
Control gene expression




Extracting signal from noise

Modeling DNA sequences

. . . GTACTCACCGGGTTACAGGATTATGGGTTACAGGTAACCGTT. . .

What can we do with a big unlabelled chunk of DNA
(e.g., parts of a newly sequenced genome) ?

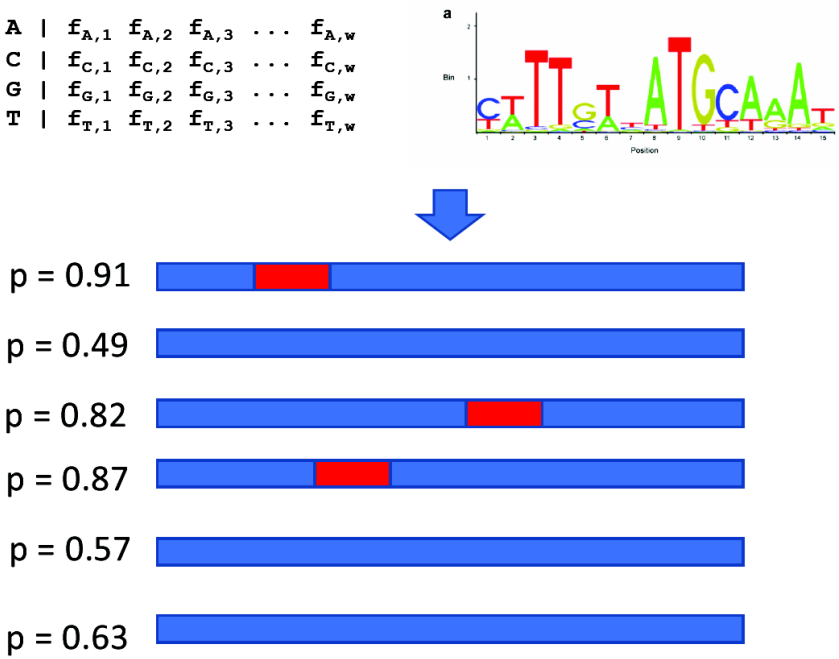
- Align it
  - to things we don't know about (assembly)
  - to things we know about (database search; e.g., known genes from related species, metagenomics)
- Analyze it's properties
  - Non-standard nucleotide composition?
  - Interesting k-mer frequencies?
  - Recurrent patterns?
- Model it
  - Make some hypotheses about it
  - Build a 'generative model' to describe it
  - Find sequences of similar type (in terms of characteristics)

$\rightarrow$ **How do we model DNA sequences?**

Modeling DNA sequences

Another more familiar example:

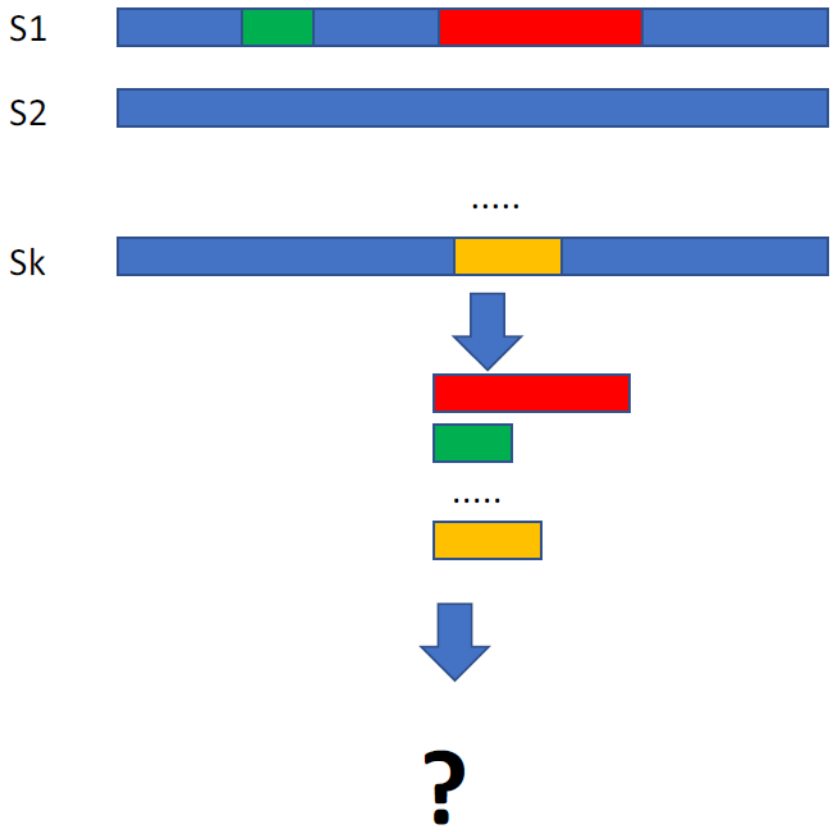
- An alignment profile/motif is a model for a specific DNA pattern (e.g., a binding site)!
- Once defined the model, we can employ it to find other instances of the pattern in new sequences
  - Do some regions in the new sequences “fit the model” well?
  - Compute overall probabilities for regions based on the frequencies (=probabilities) of individual symbols in the motif!



Modeling DNA sequences

Drawback of modeling sequences by means of profiles/motifs:

- Like in the motif, all other instances of the pattern are assumed to have the **same length** *w*
  - This works well only for certain cases, e.g., transcription factor binding sites
- How can we build a sequence model for more general cases, i.e., to **model and find interesting genomic regions of variable length?**
  - Examples of functionally relevant regions of variable length:
    - ★ CpG islands (contain a high frequency of C followed by G)
    - ★ Protein-coding genes (different numbers of exons and variable exon length!)
    - ★ ...

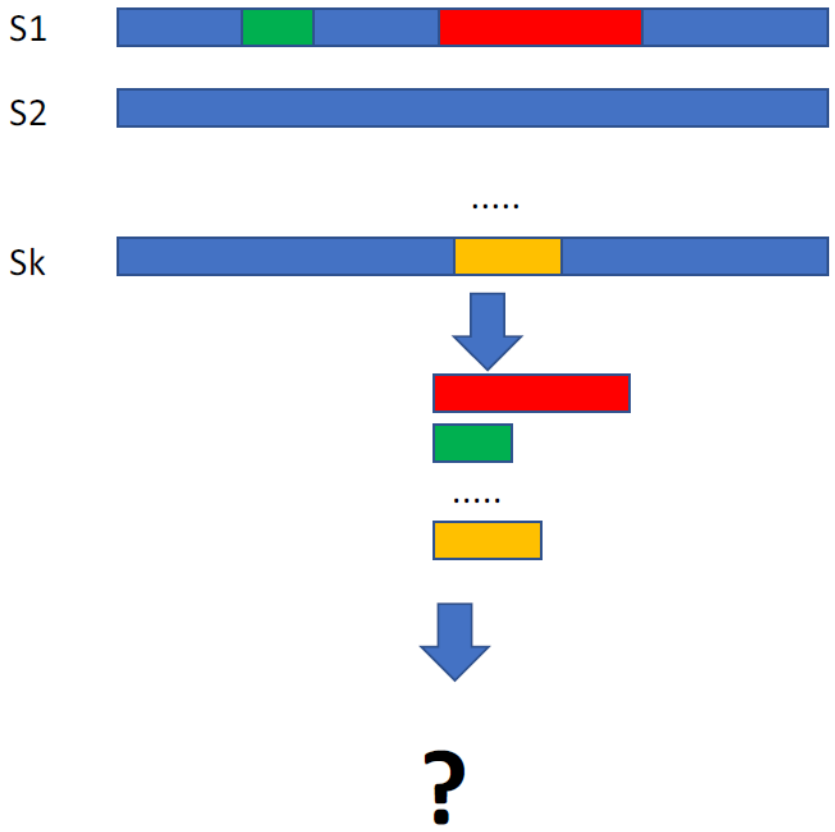


Modeling DNA sequences

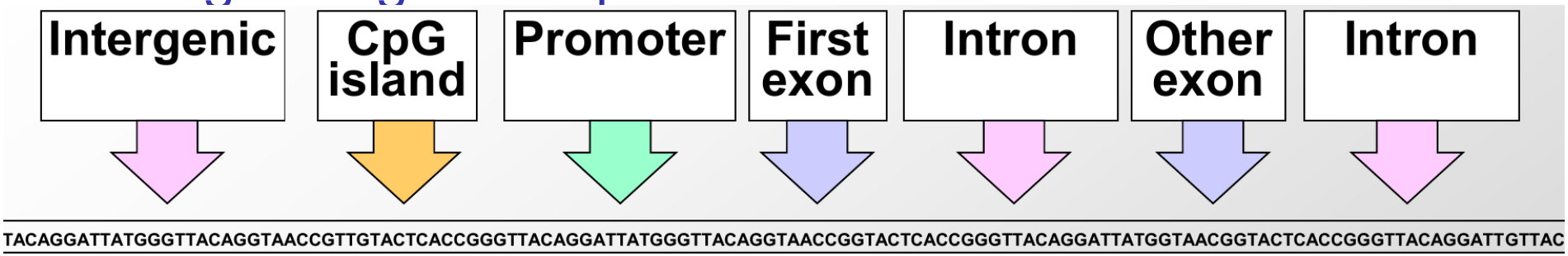
Example: difficulty of modeling and finding **protein-coding genes**

- The coding region starts with “ATG”, has a length that is a multiple of 3 (codons), and ends with a stop codon—either “TGA”, “TAA” or “TAG”
  - The total **length can be very different**
  - The coding regions are usually **interrupted by intronic regions** which are spliced out!
- Since genes can encode for very different proteins, **we can expect no sequence similarity** at all (apart from the start and stop codons)

Can we still build a model for this and use it to identify new genes in the genome?



Modeling biological sequences with HMMs



Goals of sequence modeling:

- Ability to emit (or generate) DNA sequences of a certain ***type*** (e.g., simulate promoters)
  - Not exact alignment to previously known gene, promoter, etc.
  - Preserving 'properties' of type, not identical sequence
- Ability to recognize DNA sequences of a certain ***type (state!)***
  - What (hidden) state is most likely to have generated an observation?
  - Find a set of states and transitions that generated a long sequence
- Ability to learn distinguishing characteristics of each ***state***
  - Training our generative models on large datasets
  - Learn to classify unlabelled data

Markov Chains and Hidden Markov Models

Why probabilistic sequence modeling?

- Biological data is noisy
- Probability provides a calculus for manipulating models
- Not limited to yes/no answers – can provide "degrees of belief"
- Many common computational tools are based on probabilistic models
- Our tools: **Markov Chains** and **Hidden Markov Models (HMMs)**!

Andrey Andreyevich Markov

- 1856–1922
- Russian mathematician, known for his work on stochastic processes
- "Markov" chains were the primary subject of his research



(Image from Wikipedia; public domain)

Markov Chains and Hidden Markov Models

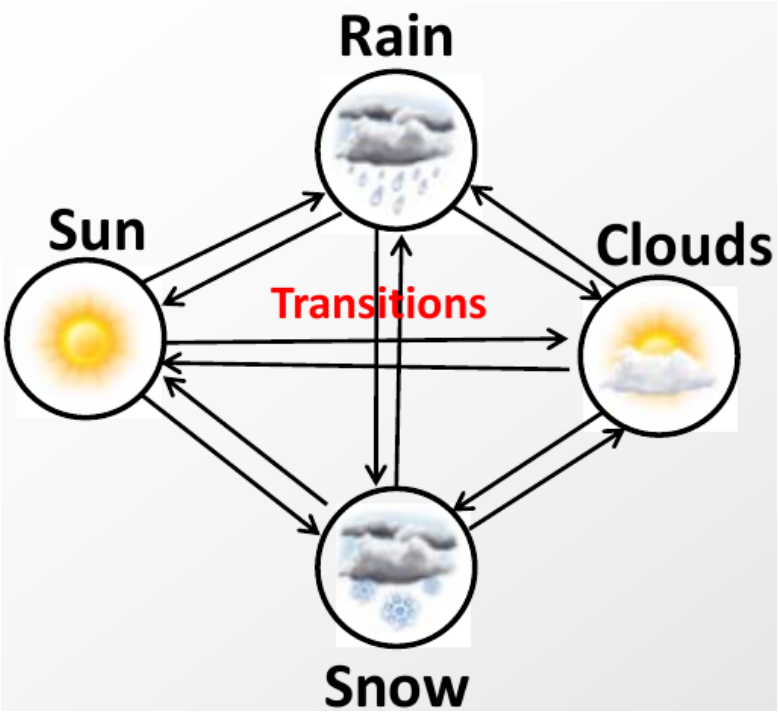
The general idea (more details later):

- **Markov Chain**
  - A sequence of data items (e.g., sequence of symbols/letters) is generated by a model (can be described by a graph) which has:
    - one "**state**" (node) per symbol/letter, and
    - certain **transition probabilities** (weighted edges) between the states

- **Hidden Markov Model (HMM)**
  - The model transitions between different states like in a Markov Chains
  - But: the **states are not directly observable ("hidden"!)**
  - Instead: each state has associated **emission probabilities** to generate certain **observable data items (symbols)**
    - That is: the observed sequence of data items was generated by the model transition through different states which are, however, not directly observable
    - The set of hidden states is usually not identical/equivalent to the set of emitted symbols/letters!
  - In other words: **we don't know in which state the model is in, we only observe which characters the model is emitting!**

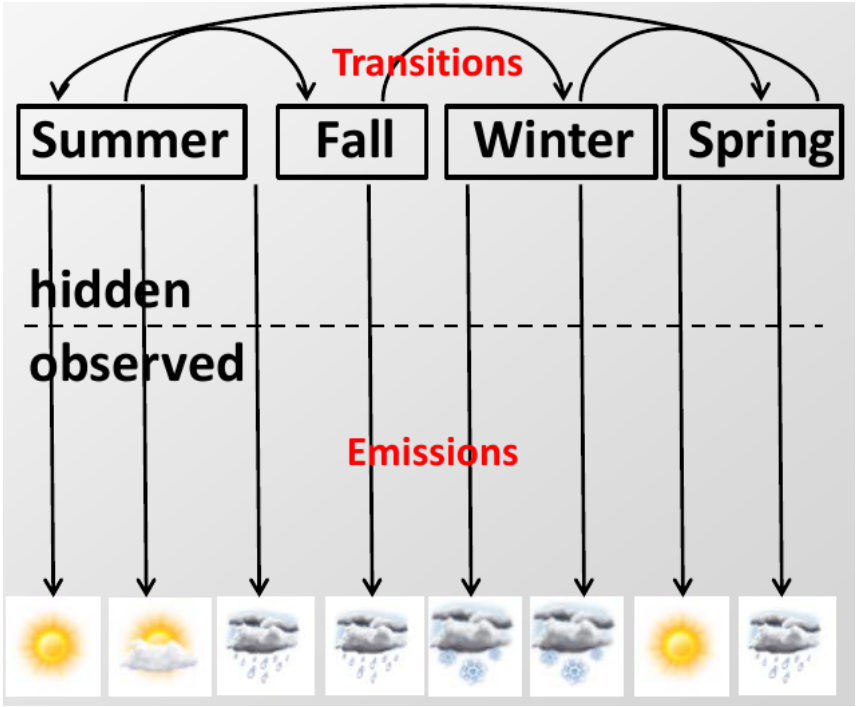
Predicting tomorrow's weather

Markov chain



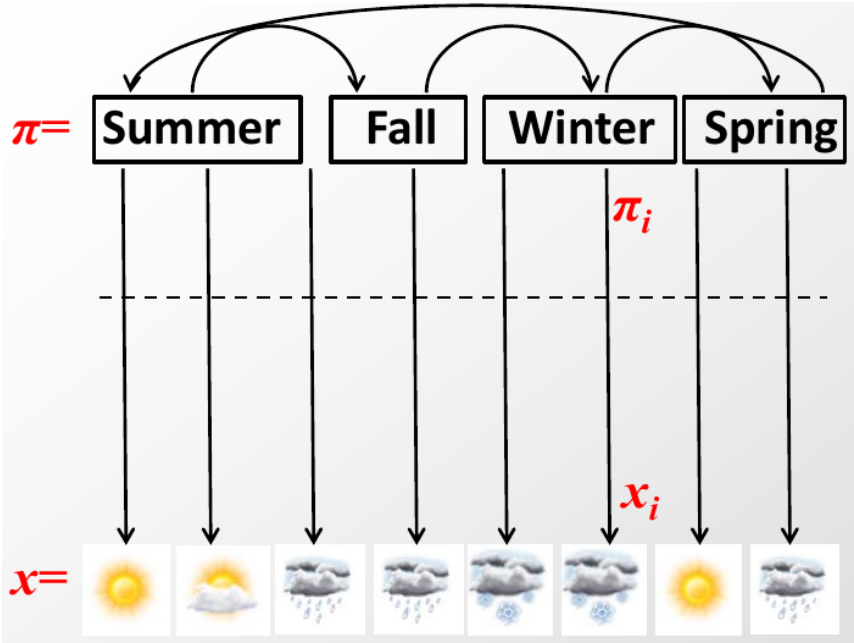
- What you see is what you get (**states are directly observed**)
- Next state only depends on current state (**no memory**!)

Hidden Markov Model (HMM)



- **Hidden state** (e.g., season, storm system) determines probabilities of **observations** (e.g., weather)
- State transitions are governed by a Markov chain

HMM nomenclature for this course



Transitions: $a_{kl} = P(\pi_i = l | \pi_{i-1} = k)$

- **Transition probability** from hidden state k to hidden state l

Emissions: $e_k(x_i) = P(x_i | \pi_i = k)$

- **Emission probability** of symbol x_i when the hidden state is k

- Vector $\pi = (\pi_1, \pi_2, \dots)$: **hidden path** (sequence of hidden states of a set Σ)
- Vector $x = (x_1, x_2, \dots)$: sequence of **observations** of a set V of m symbols
- **Transition matrix** $A = (a_{kl})$: probability of $k \to l$ state transitions
- **Emission vector** $E_k = \big(e_k(v_1), \dots, e_k(v_m)\big)$: prob. of observing the possible observable letters $v_j \in V$ from state k
  - **Emission matrix**: contains emission probabilities for all states
- Bayes' rule: use $P(x_i | \pi_i = k)$ to estimate $P(\pi_i = k | x_i)$

Components of a Markov chain

Definition: A *Markov chain* is a triplet (Q, p, A), where:

- Q is a finite set of states; each state corresponds to a symbol in the alphabet Σ
- p is a vector of initial state probabilities
- A is the matrix of state transition probabilities, denoted by a_{kl} for each $k, l \in Q$
  - For each $k, l \in Q$ the transition probability is: $a_{kl} \equiv P(x_i = l | x_{i-1} = k)$

Output: The output of the model is the state at each instant time (path) $\rightarrow$ the *states are directly observable*

Property: The probability of each observed symbol x_i (= state $q \in Q$ at time step i) depends only on the value of the preceding symbol x_{i-1}:

$$P(x_i | x_{i-1}, \ldots, x_1) = P(x_i | x_{i-1})$$

- In other words: *transition probabilities to the next state depend only on the current state*, not on the previous path that led to it (no memory)!

Formula: The probability of observing a specific sequence x of length L is:

$$P(x) = P(x_L, x_{L-1}, \ldots, x_1) = P(x_L | x_{L-1}) \times P(x_{L-1} | x_{L-2}) \times \cdots \times P(x_2 | x_1) \times P(x_1)$$

where $P(x_1)$ is the initial state probability of x_1

(additional slide credit: Serafim Batzoglou)

Components of a Hidden Markov Model (HMM)

Definition: An HMM is a 5-tuple (Q, V, p, A, E), where:

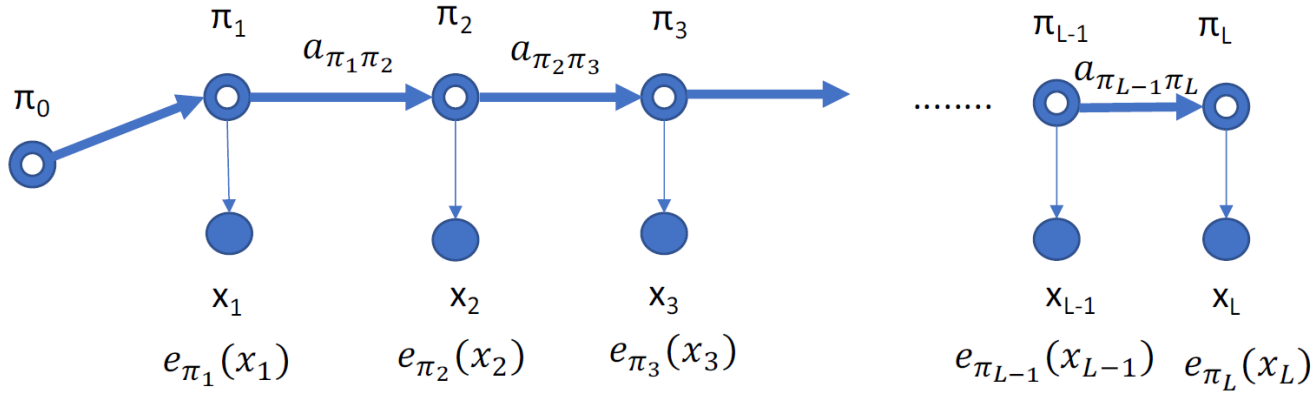
- Q is a finite set of states q_n; $|Q| = N$
- V is a finite set of observable symbols v_m; $|V| = M$
- p is a vector of initial state probabilities
- A is the matrix of state transition probabilities, denoted by a_{kl} for each $k, l \in Q$
  - For each $k, l \in Q$ the transition probability is: $a_{kl} \equiv P(x_i = l | x_{i-1} = k)$
- E is an $N \times M$ emission probability matrix,
  $e_{nm} \equiv P(x_i = v_m \in V \text{ at step } i | \pi_i = q_n \in Q)$
  - Note: e_{nm} does not depend on the time step i, i.e., emission probabilities of the symbols $v_m \in V$ depend only on the states $q_n \in Q$ and don't change over time!

Output: Only *emitted symbols* from V are *observable* by the system but not the underlying random walk between states from $Q \rightarrow$ "hidden"!

Property: Both *emissions and transitions are dependent on the current state only* and not on the past (no memory)!

(additional slide credit: Serafim Batzoglou)

HMM: hidden path and emissions



A typical representation of a process which can be modeled by an HMM:

- Start: a **random inital state** $\pi_1 \in Q$ according to the initial state probabilities p
  - Note: in this depiction, state "π_0" represents a sort of "null" state before the process starts; the transition from π_0 ot π_1 represents the choice of the first/initial state (π_1)
- The **random choice to move** from the current (hidden) state $\pi_{i-1} = k \in Q$ to the next (hidden) state $\pi_i = l \in Q$ depends on transition probabilities a_{kl}
  - The set of possible random choices also includes $k \to k$; so if $a_{kk} \neq 0$, it is possible that $\pi_i = \pi_{i-1}$, i.e., the system remains in the same state
- For each step i of the hidden path, the hidden state π_i **randomly emits an observable symbol** $x_i \in V$ according to the emission probabilities $e_{\pi_i}(x_i)$
  - For example, let the current hidden state be "summer"; this defines the probabilities of observing different weather: "sun", "clouds", "rain", "snow", ...
  - For the hidden state "winter", instead, the same weather "symbols" can be observed, but with significantly different "emission" probabilities!

The six algorithmic settings for HMMs

	One path	All paths
Scoring	1. Scoring x, one path $P(x,\pi)$ Prob of a path, emissions	2. Scoring x, all paths $P(x) = \sum_\pi P(x,\pi)$ Prob of emissions, over all paths
Decoding	3. Viterbi decoding $\pi^* = \text{argmax}_\pi P(x,\pi)$ Most likely path	4. Posterior decoding $\pi\text{^} = \{\pi_i \mid \pi_i = \text{argmax}_k \sum_\pi P(\pi_i=k\mid x)\}$ Path containing the most likely state at any time point.
Learning	5. Supervised learning, given π $\Lambda^* = \text{argmax}_\Lambda P(x,\pi\mid\Lambda)$ 6. Unsupervised learning. $\Lambda^* = \text{argmax}_\Lambda \max_\pi P(x,\pi\mid\Lambda)$ Viterbi training, best path	6. Unsupervised learning $\Lambda^* = \text{argmax}_\Lambda \sum_\pi P(x,\pi\mid\Lambda)$ Baum-Welch training, over all paths

Examples of HMMs

The dishonest casino

The dishonest genome

... (and many more)

Example: the dishonest casino

A **hypothetical casino** has two dice:

- Fair die D_F:
  $P(1) = P(2) = P(3) = P(4) = P(5) = P(6) = 1/6$

- Loaded (unfair) die D_L:
  $P(1) = P(2) = P(3) = P(4) = P(5) = 1/10$
  $P(6) = 1/2$

Casino player switches between fair and loaded die on average once every 20 turns, i.e., $P(D_F \rightarrow D_L) = P(D_L \rightarrow D_F) = 1/20$

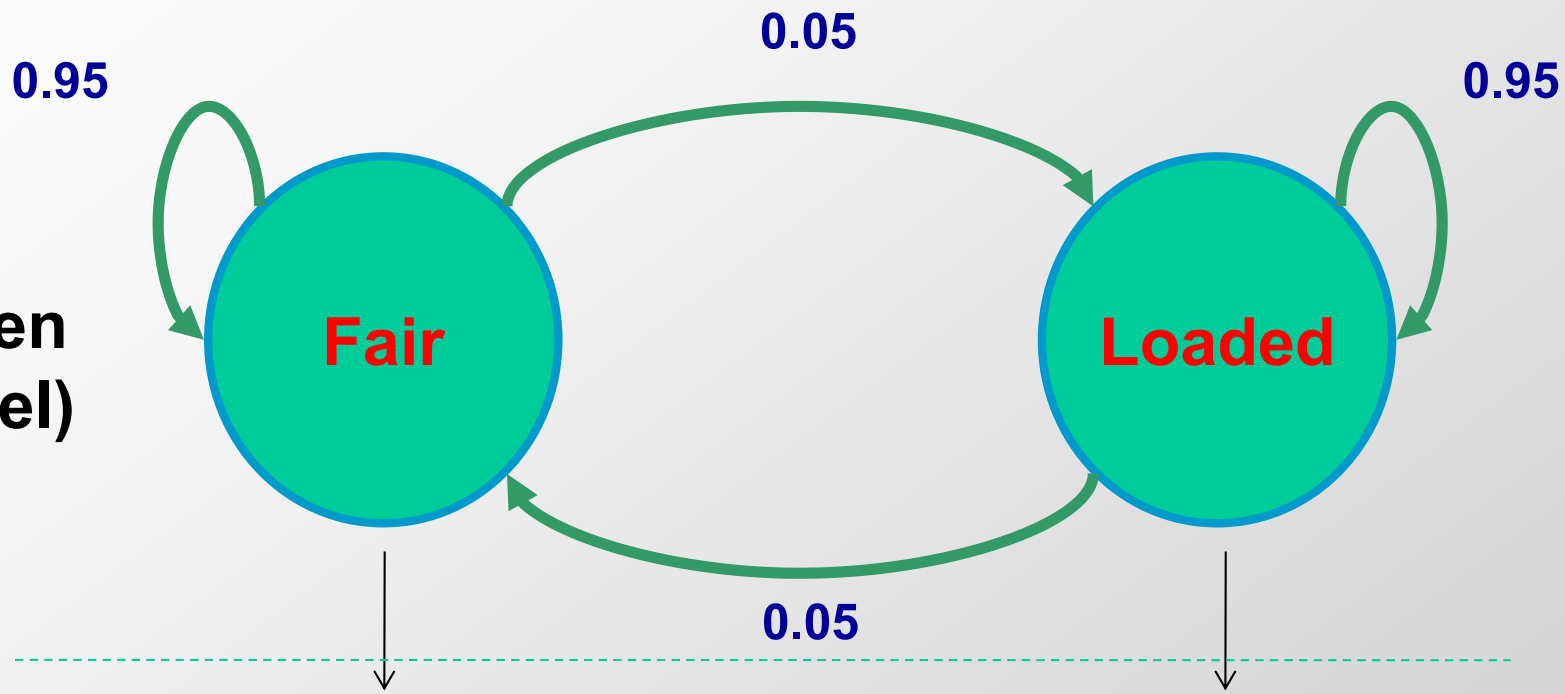
[Thus: $P(D_F \rightarrow D_F) = P(D_L \rightarrow D_L) = 19/20$]

Game:

1. You bet \$1
2. You roll (always with a fair die)
3. Casino player rolls (maybe with a fair die, maybe with a loaded die)
4. Highest number wins \$2

(additional slide credit: Serafim Batzoglou)

The dishonest casino model



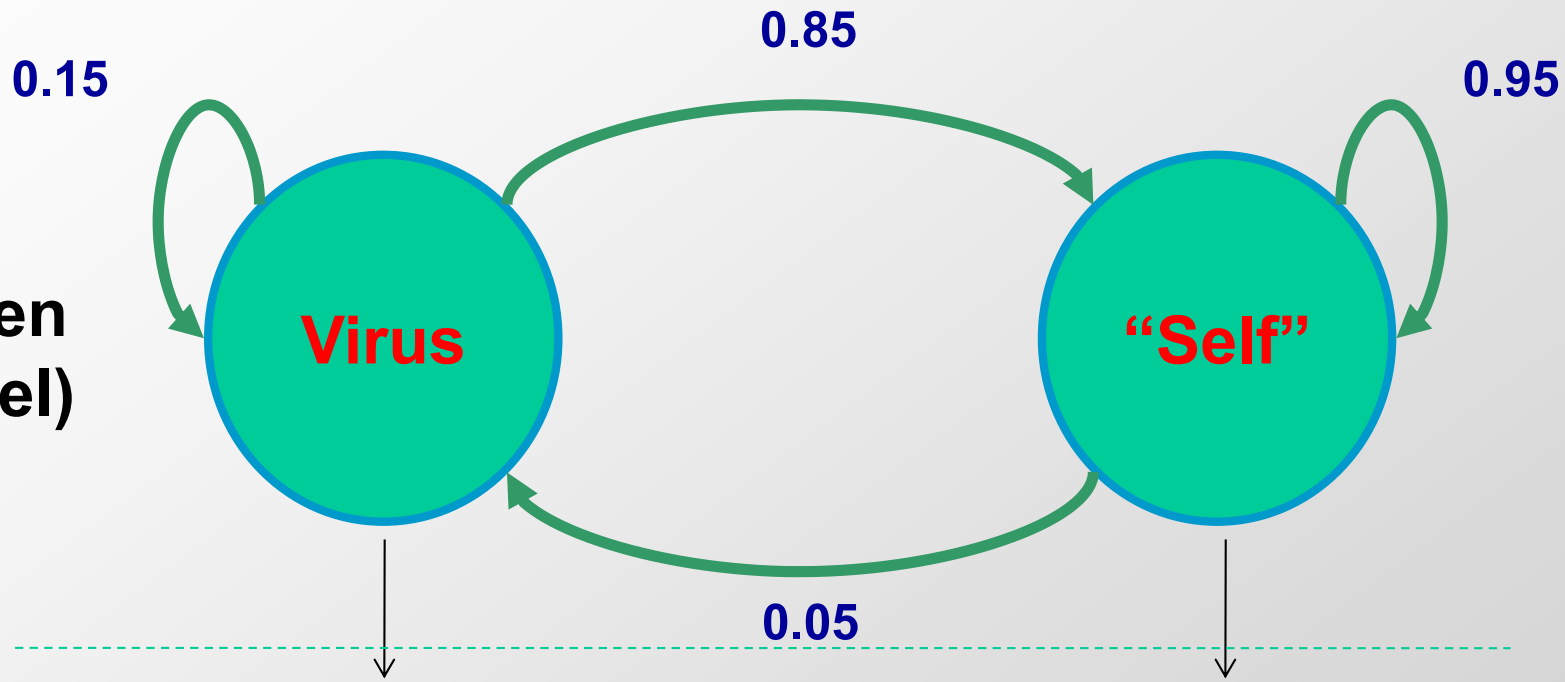
Hidden (model)

Observed (world)

Fair	Loaded
P(1|Fair) = 1/6	P(1|L) = 1/10
P(2|Fair) = 1/6	P(2|L) = 1/10
P(3|Fair) = 1/6	P(3|L) = 1/10
P(4|Fair) = 1/6	P(4|L) = 1/10
P(5|Fair) = 1/6	P(5|L) = 1/10
P(6|Fair) = 1/6	P(6|L) = 1/2

Slide credit: Serafim Batzoglou

The dishonest genome model



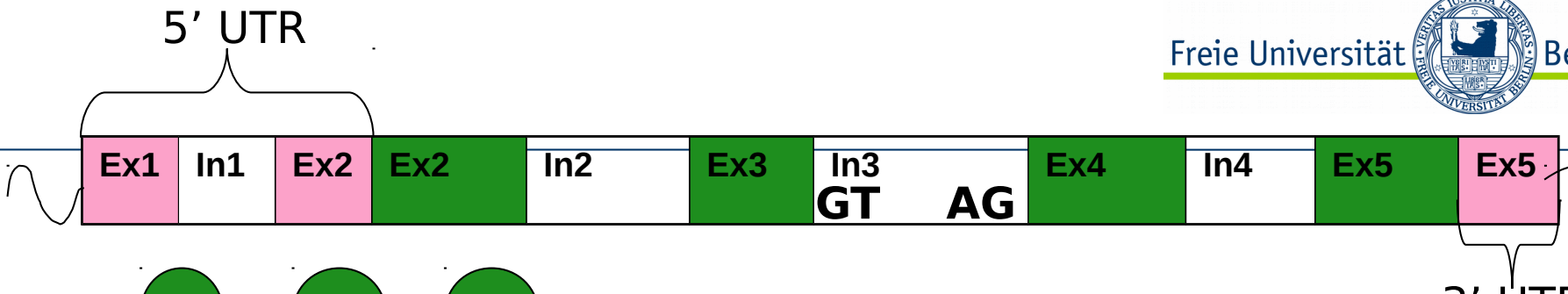
Hidden (model)

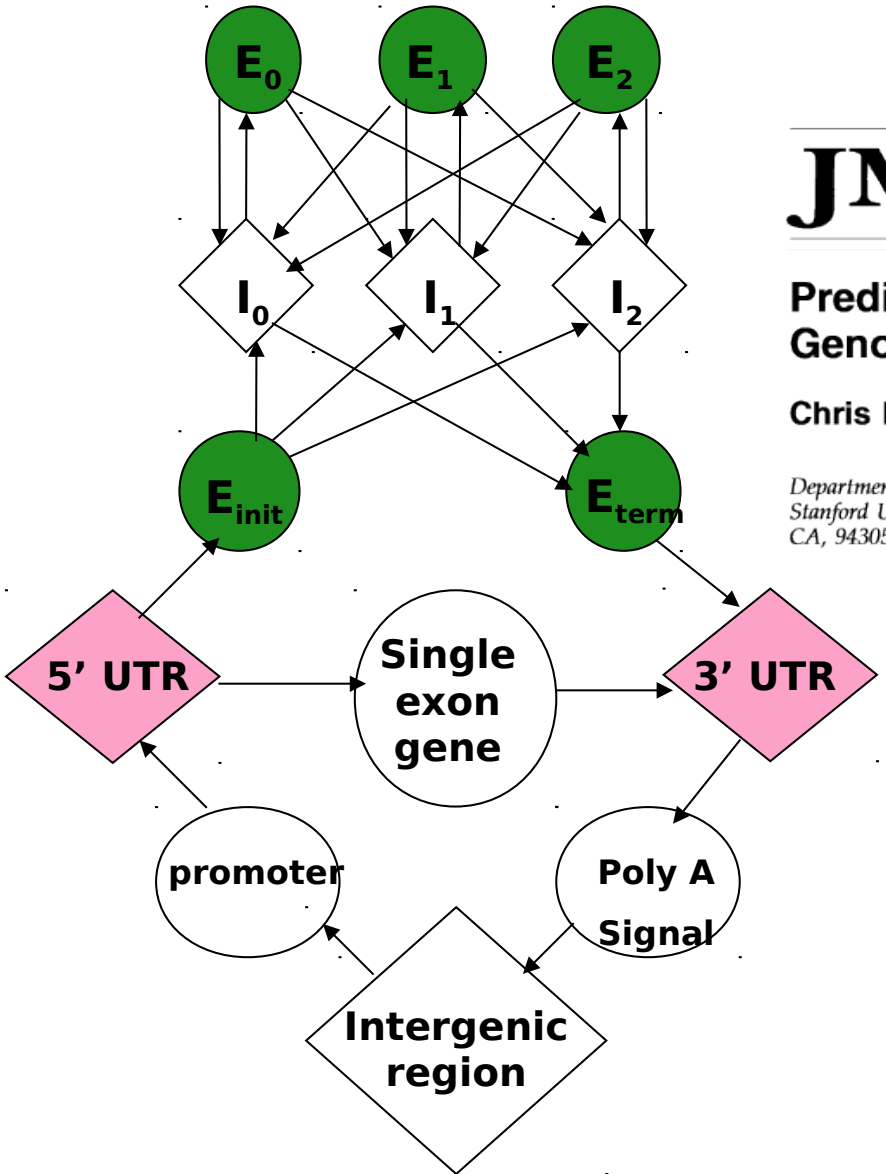
Observed (world)

P(A|Virus) = 1/6
P(T|Virus) = 1/6
P(C|Virus) = 1/3
P(G|Virus) = 1/3

P(A|Self) = 1/4
P(T|Self) = 1/4
P(C|Self) = 1/4
P(G|Self) = 1/4

Slide credit: Serafim Batzoglou





J. Mol. Biol. (1997) 268, 78–94

JMB

Prediction of Complete Gene Structures in Human Genomic DNA

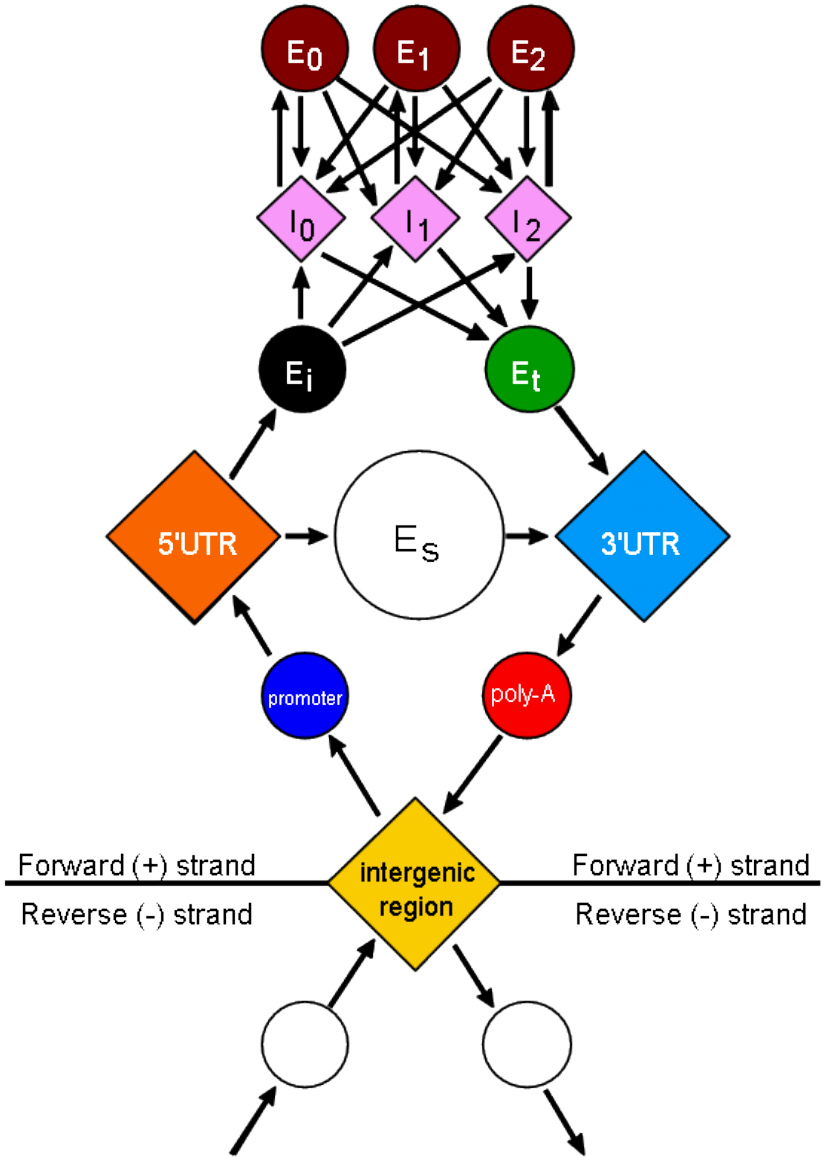
Chris Burge* and Samuel Karlin

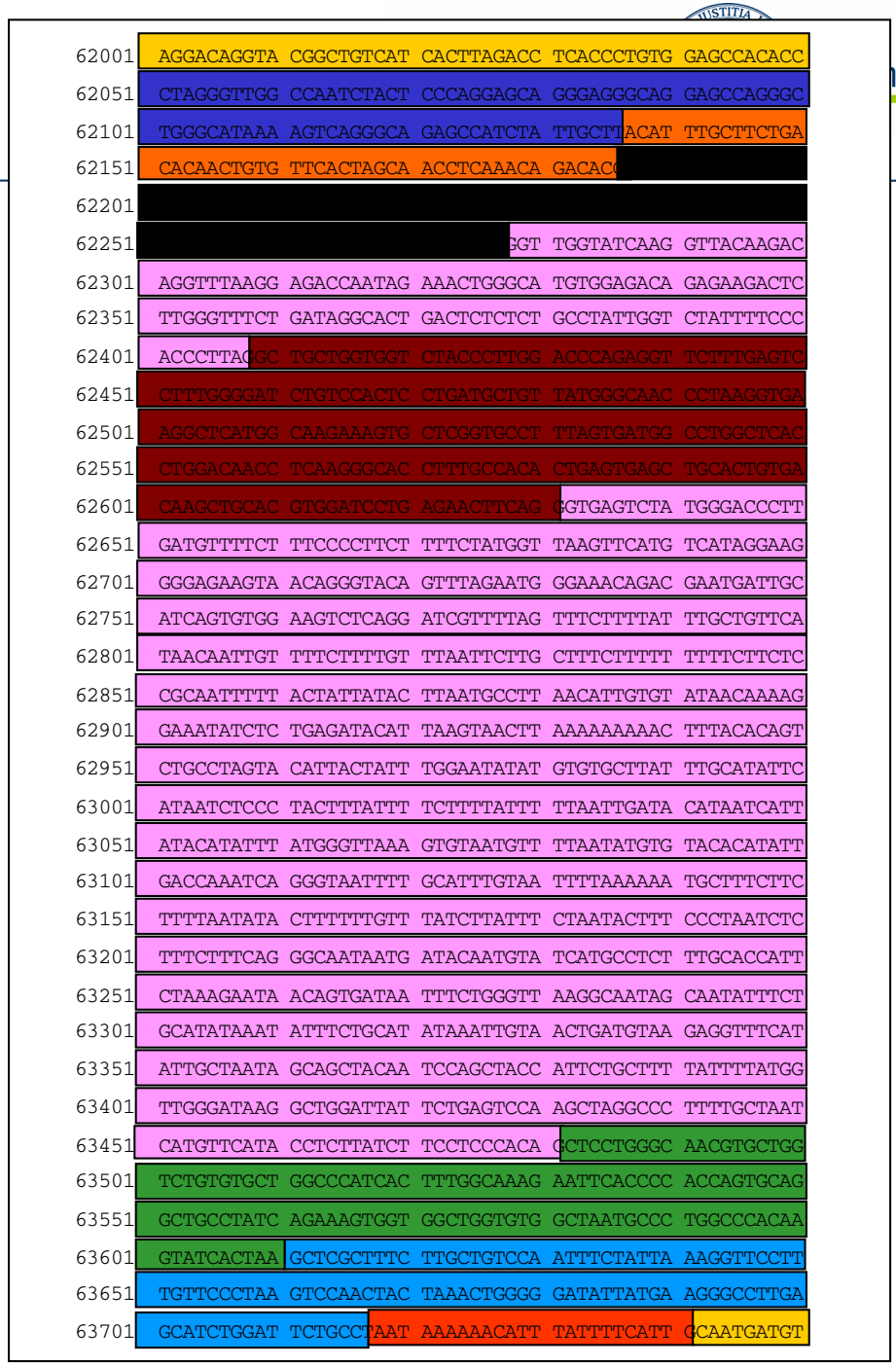
Department of Mathematics Stanford University, Stanford CA, 94305, USA

We introduce a general probabilistic model of the gene structure of human genomic sequences which incorporates descriptions of the basic transcriptional, translational and splicing signals, as well as length distributions and compositional features of exons, introns and intergenic regions. Distinct sets of model parameters are derived to account for the many substantial differences in gene density and structure observed in distinct C + G compositional regions of the human genome. In addition, new models of the donor and acceptor splice signals are described which capture potentially important dependencies between signal positions. The model is applied to the problem of gene identification in a computer program, GENSCAN, which identifies complete exon/intron structures of genes in genomic DNA. Novel features of the program include the ca-

Slide by Ron Pinter

GENSCAN (Burge & Karlin)

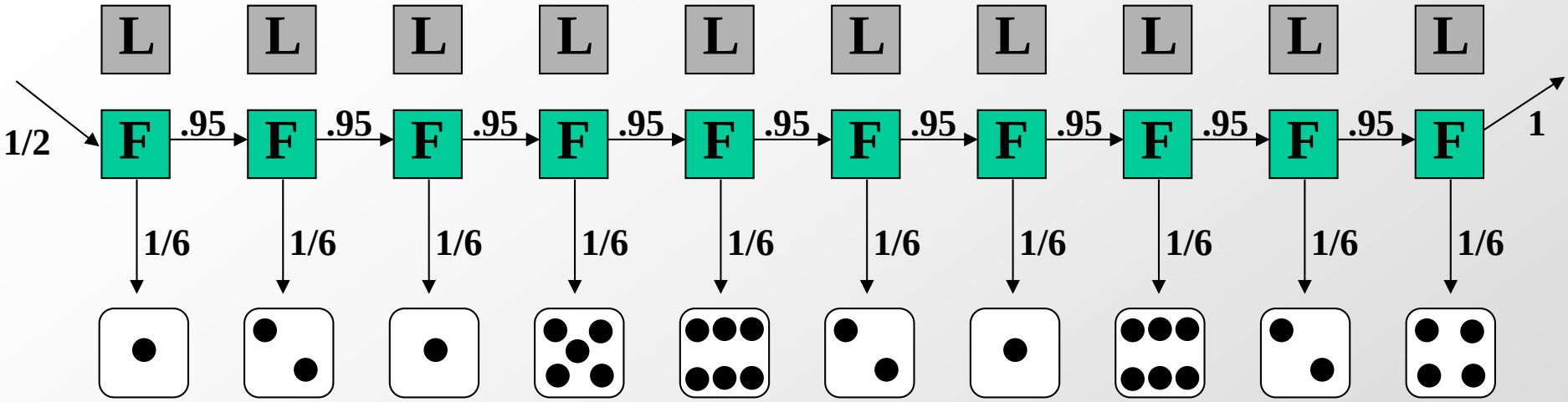




Examples of HMMs for genome annotation

Application	Detection of GC-rich regions	Detection of conserved regions	Detection of protein-coding exons	Detection of protein-coding conservation	Detection of protein-coding gene structures	Detection of chromatin states
Topology / Transitions	2 states, different nucleotide composition	2 states, different conservation levels	2 states, different tri-nucleotide composition	2 states, different evolutionary signatures	~20 states, different composition/ conservation, specific structure	40 states, different chromatin mark combinations
Hidden States / Annotation	GC-rich / AT-rich	Conserved / non-conserved	Coding exon / non-coding (intron or intergenic)	Coding exon / non-coding (intron or intergenic)	First/last/middle coding exon,UTRs, intron1/2/3, intergenic, *(+/- strand)	Enhancer / promoter / transcribed / repressed / repetitive
Emissions / Observations	Nucleotides	Level of conservation	Triplets of nucleotides	Nucleotide triplets, conservation levels	Codons, nucleotides, splice sites, start/stop codons	Vector of chromatin mark frequencies

Running the model: Probability of a sequence



What is the joint probability of observing x and a specific path π:

π = Fair, Fair, Fair, Fair, Fair, Fair, Fair, Fair, Fair, Fair

and rolls

x = 1 , 2, 1, 5, 6, 2, 1, 6, 2, 4

Joined probability $P(x,\pi)=P(x|\pi)P(\pi)=P(\text{emissions}|\text{path})*P(\text{path})$

emission transition emission transition emission

$p = ½ \times P(1 \mid Fair)\ P(Fair_{i+1} \mid Fair_i)\ P(2 \mid Fair)\ P(Fair \mid Fair) \ldots P(4 \mid Fair)$

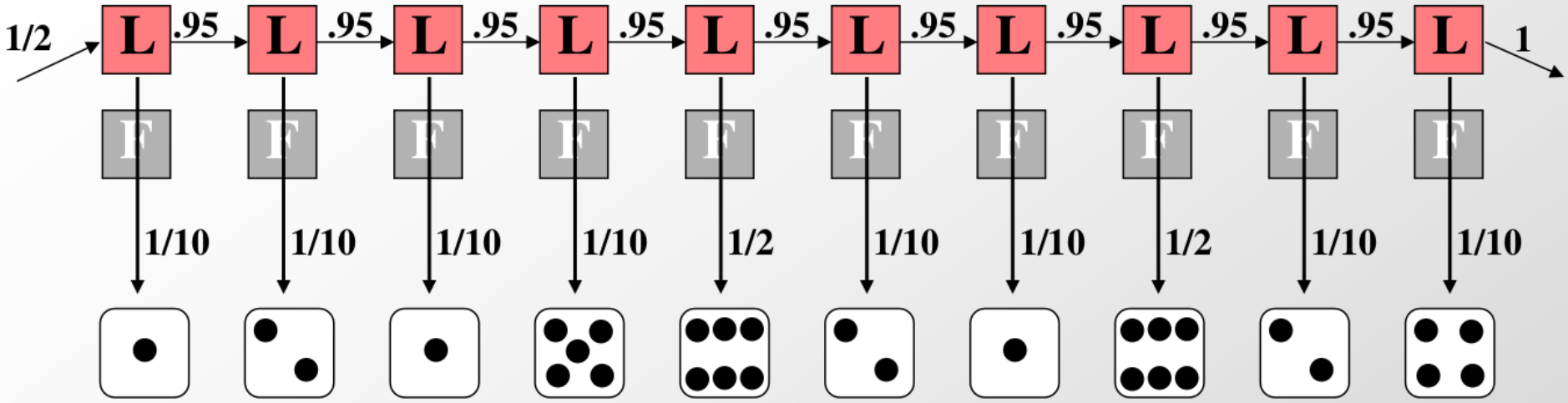
$= ½ \times (1/6)^{10} \times (0.95)^9$

$= 5.2 \times 10^{-9}$

Why is p so small?

Slide credit: Serafim Batzoglou

Running the model: Probability of a sequence



What is the likelihood of

π = Load, Load, Load, Load, Load, Load, Load, Load, Load, Loaded

and rolls

x = 1 , 2, 1, 5, 6, 2, 1, 6, 2, 4

emission transition emission transition emission

$p = ½ \times P(1 \mid \text{Load}) P(\text{Load}_{i+1} \mid \text{Load}_i) P(2 \mid \text{Load}) P(\text{Load}|\text{Load}) \ldots P(4 \mid \text{Load})$

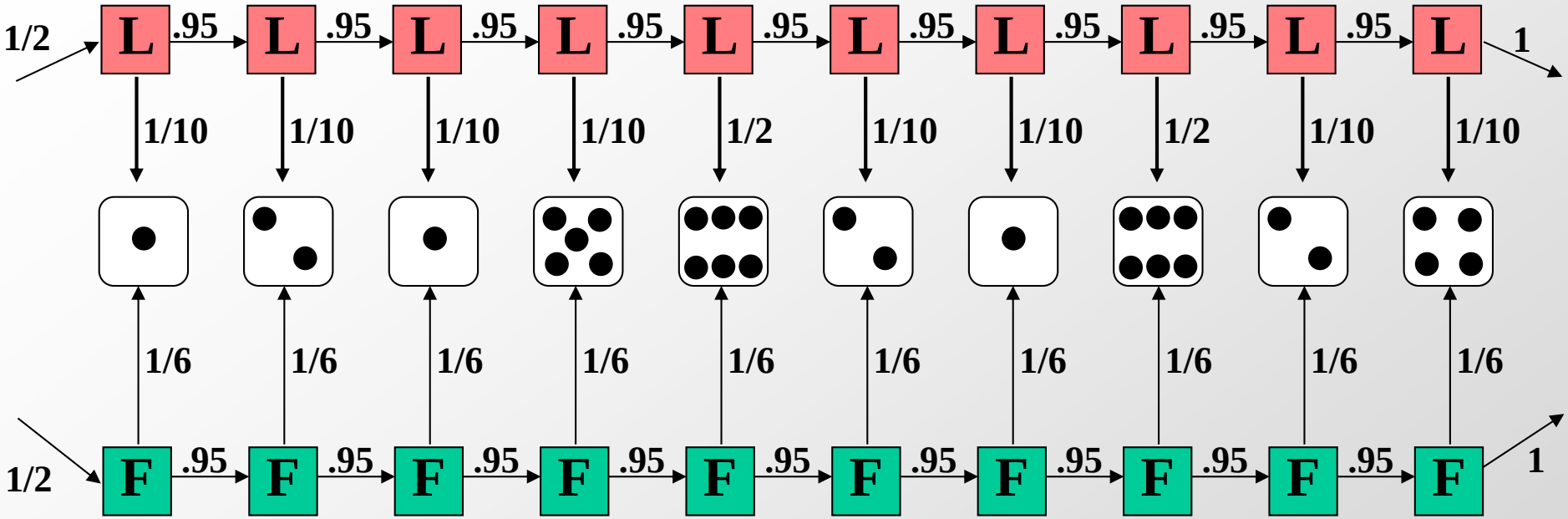
$= ½ \times (1/10)^8 \times (1/2)^2 (0.95)^9$

$= 7.9 \times 10^{-10}$

Compare the two!

Slide credit: Serafim Batzoglou

Comparing the two paths



Two sequence paths:

P(x, all-Fair) = 5.2×10^{-9} (very small)

P(x, all-Loaded) = 7.9×10^{-10} (very very small)

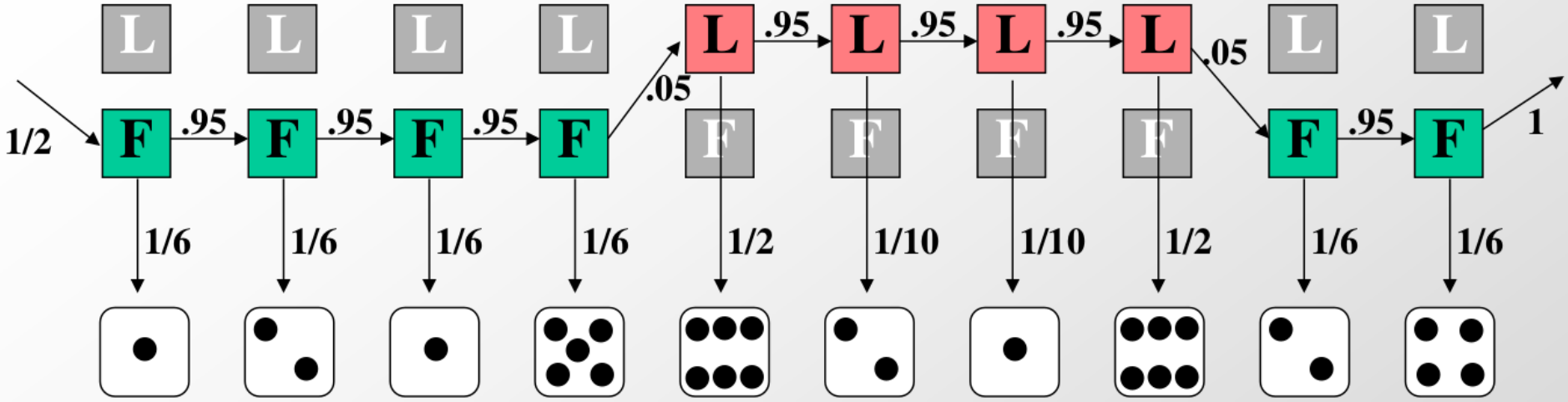
Likelihood ratio:

P(x, all-Fair) is 6.59 times more likely than P(x, all-Loaded)

It is 6.59 times more likely that the die is fair all the way, than loaded all the way.

Slide credit: Serafim Batzoglou

What about partial runs and die switching



What is the likelihood of

π = Fair, Fair, Fair, Fair, Load, Load, Load, Load, Fair, Fair

and rolls

x = 1 , 2, 1, 5, 6, 2, 1, 6, 2, 4

emission transition emission transition emission

$p = ½ \times P(1 \mid Fair)\, P(Fair_{i+1} \mid Fair_i)\, P(2 \mid Fair)\, P(Fair \mid Fair) \ldots P(4 \mid Fair)$

$= ½ \times (1/10)^2 \times (1/2)^2 \times (1/6)^6 \times (0.95)^7 \times (0.05)^2$

$= 4.7 \times 10^{-11}$

Much less likely, due to high cost of transitions

Model comparison

Let the sequence of rolls be:

x = 1, 6, 6, 5, 6, 2, 6, 6, 3, 6

Now, what is the likelihood π = F, F, …, F?

$\frac{1}{2} \times (1/6)^{10} \times (0.95)^{9} = 0.5 \times 10^{-9}$, same as before

What is the likelihood π = L, L, …, L?

$\frac{1}{2} \times (1/10)^{4} \times (1/2)^{6} (0.95)^{9} = 0.5 \times 10^{-7}$

So, it is 100 times more likely the die is loaded

Model evaluation

The six algorithmic settings for HMMs

	One path	All paths
Scoring	1. Scoring x, one path $P(x,\pi)$ Prob of a path, emissions ✓	2. Scoring x, all paths $P(x) = \sum_\pi P(x,\pi)$ Prob of emissions, over all paths
Decoding	3. Viterbi decoding $\pi^* = \text{argmax}_\pi P(x,\pi)$ Most likely path ★	4. Posterior decoding $\pi\hat{} = \{\pi_i \mid \pi_i = \text{argmax}_k \sum_\pi P(\pi_i=k\mid x)\}$ Path containing the most likely state at any time point.
Learning	5. Supervised learning, given π $\Lambda^* = \text{argmax}_\Lambda P(x,\pi\mid\Lambda)$ 6. Unsupervised learning. $\Lambda^* = \text{argmax}_\Lambda \max_\pi P(x,\pi\mid\Lambda)$ Viterbi training, best path	6. Unsupervised learning $\Lambda^* = \text{argmax}_\Lambda \sum_\pi P(x,\pi\mid\Lambda)$ Baum-Welch training, over all paths

3. DECODING:
What was the sequence of hidden states?

Given: Model parameters $e_i(.)$, a_{ij}

Given: Sequence of emissions x

Find: Sequence of hidden states π

Finding the optimal path

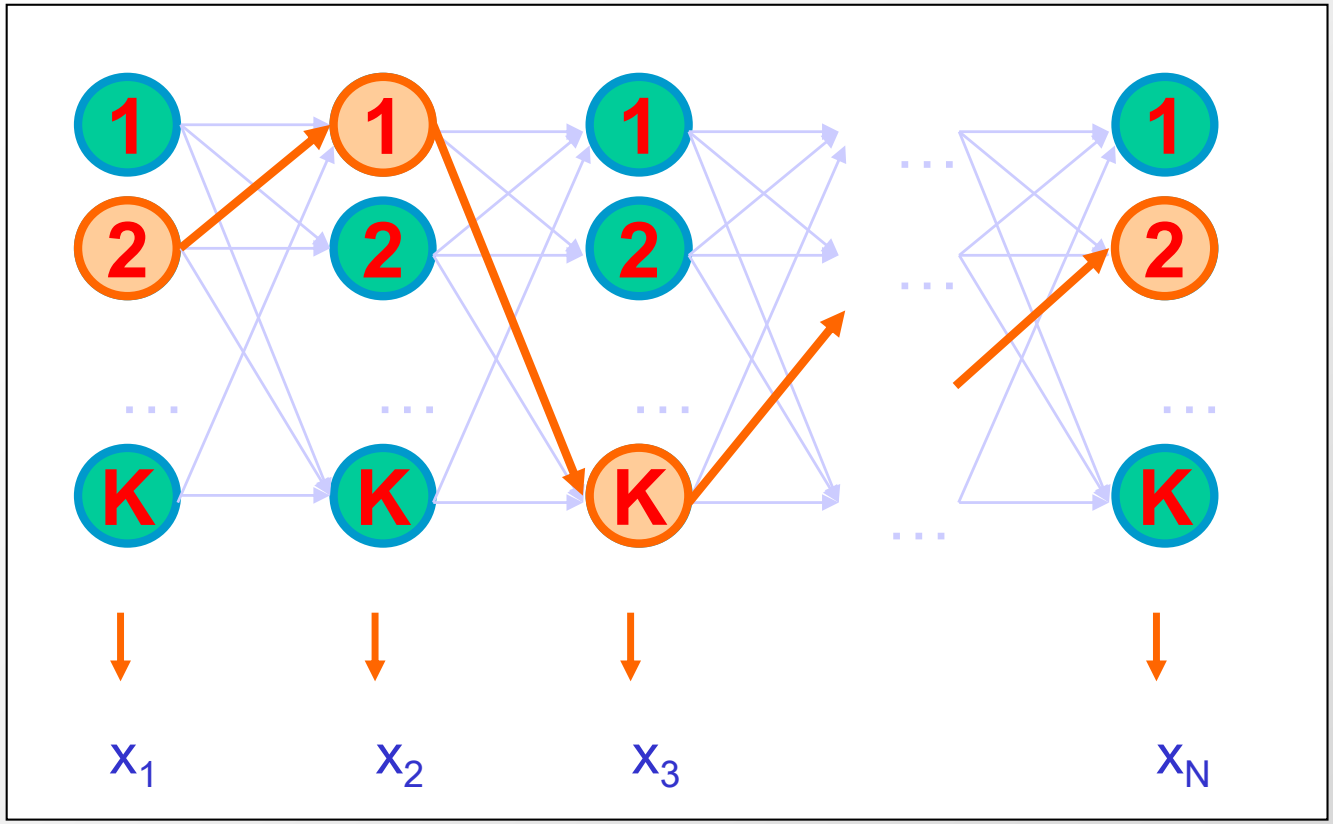
- We can now evaluate any path through hidden states, given the emitted sequences
- How do we find the best path?
- Optimal substructure! Best path through a given state is:
  - Best path to previous state
  - Best transition from previous state to this state
  - Best path to the end state

➔ Viterbi algortithm

- Define $V_k(i)$ = Probability of the most likely path through state $\pi_i=k$
- Compute $V_k(i+1)$ as a function of $\max_{k'} \{ V_{k'}(i) \}$
- $V_k(i+1) = e_k(x_{i+1}) * \max_j a_{jk} V_j(i)$

➔ Dynamic Programming

Finding the most likely path

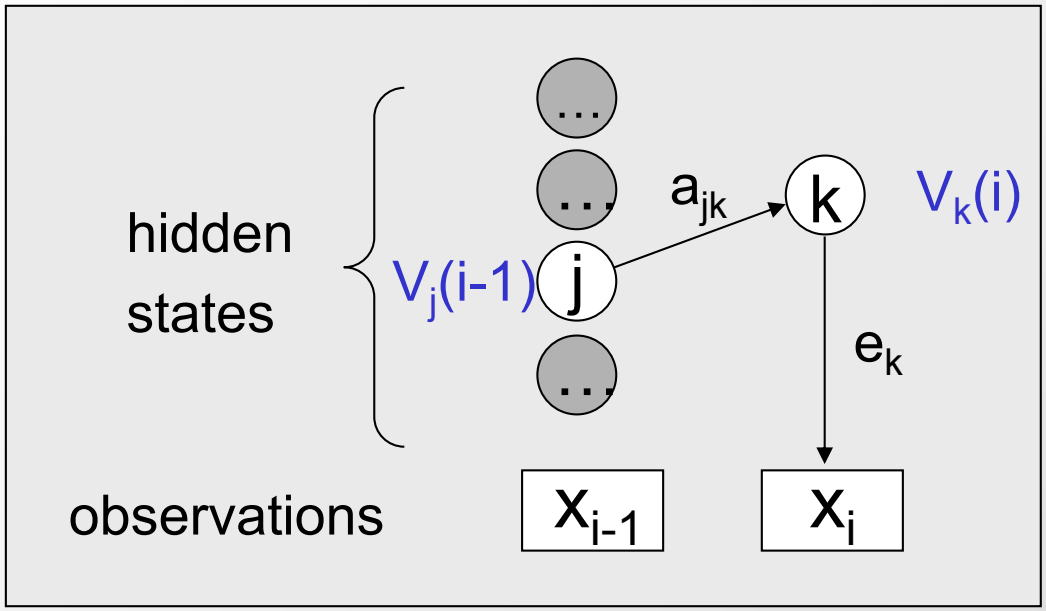


- Find path π^* that maximizes total joint probability $P[x, \pi]$
  - $P(x,\pi) = a_{0\pi_1} * \prod_i e_{\pi_i}(x_i) \times a_{\pi_i\pi_{i+1}}$

start emission transition

Slide credit: Serafim Batzoglou

Calculate maximum P(x,π) recursively



- Assume we know V_j for the previous time step (i-1)
- Calculate $V_k(i) = e_k(x_i) * \max_j (V_j(i-1) \times a_{jk})$

$V_k(i)$: current max

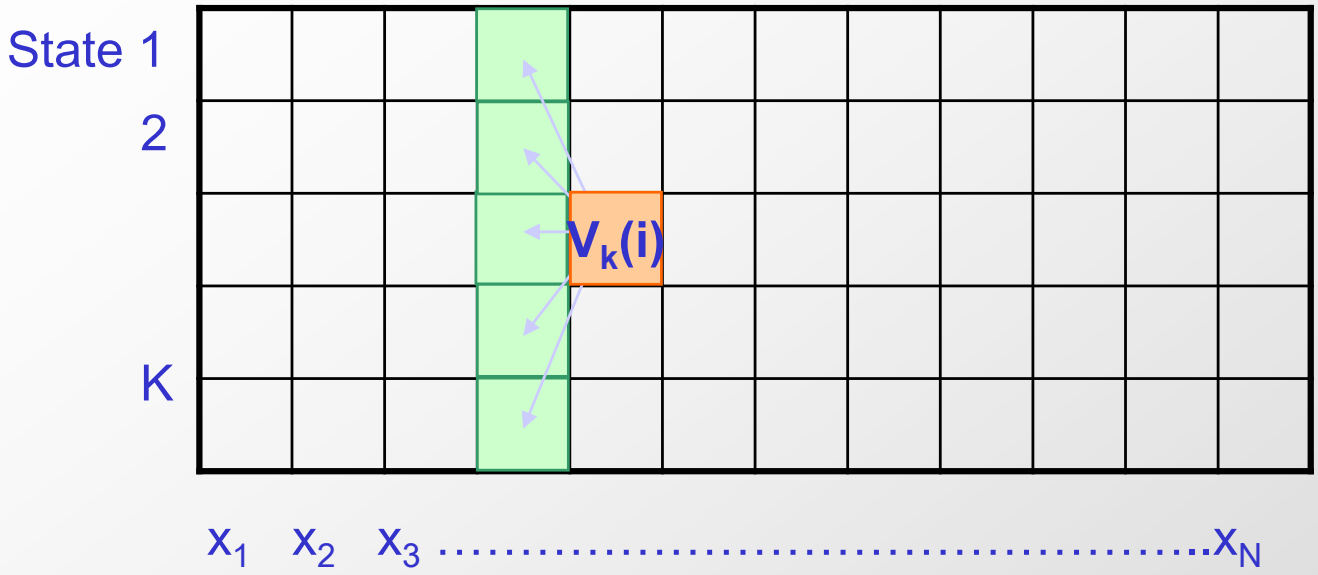
$e_k(x_i)$: this emission

$V_j(i-1)$: max ending in state j at step i

a_{jk}: Transition from state j

all possible previous states j

The Viterbi Algorithm



Input: x = x1……xN

Initialization:

$V_0(0)=1$, $V_k(0) = 0$, for all $k > 0$

Iteration:

$V_k(i) = e_K(x_i) \times \max_j a_{jk} V_j(i-1)$

Termination:

$P(x, \pi^*) = \max_k V_k(N)$

Traceback:

Follow max pointers back

Similar to aligning states to seq

In practice:

Use log scores for computation

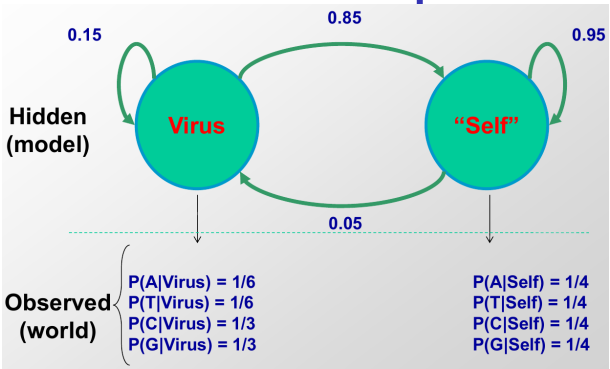
Running time and space:

Time: $O(K^2N)$

Space: $O(KN)$

Slide credit: Serafim Batzoglou

Viterbi: example



Step	1	2	3	4	5
Emission	G	C	G	A	T
State V=Virus	$V_V(1)$ = 0.5*0.33 = 0.165				
State S="Self"	$V_S(1)$ = 0.5*0.25 = 0.125				

Assume the following initial state probabilities (from "null" to V or S):

- $p(V) = a_{0V} = 0.5$
- $p(S) = a_{0S} = 0.5$

Initialization (Step i=0):

- $V_0(0) = 1$; $V_V(0) = 0$; $V_S(0) = 0$
  That is, at the beginning we are in a "null" state, not in V or S!

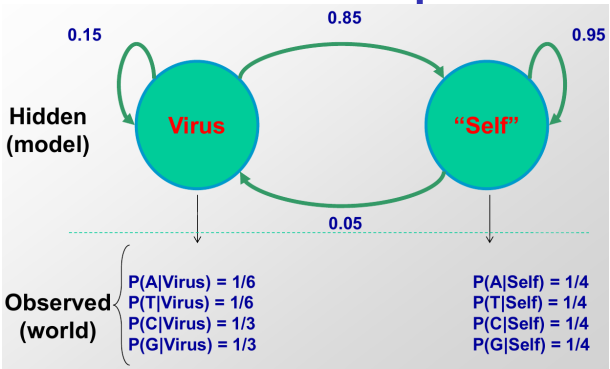
Iterative procedure:

- $V_k(i) = e_k(x_i) \times \max_j a_{jk} V_j(i-1)$

Step i=1: fill column 1

- $V_v(1) = e_V('G') \times a_{0V} V_0(0) = 0.33 \times 0.5 * 1 = 0.165$
- $V_S(1) = e_S('G') \times a_{0S} V_0(0) = 0.25 \times 0.5 * 1 = 0.125$

Viterbi: example



Step	1	2	3	4	5
Emission	G	C	G	A	T
State V=Virus	$V_V(1)$ = 0.5*0.33 = 0.165	$V_V(2)$ = 0.33*0.15*$V_V(1)$ = $8.1*10^{-3}$	$V_V(3)$ = 0.33*0.05*$V_S(2)$ = $5.8*10^{-4}$		
State S="Self"	$V_S(1)$ = 0.5*0.25 = 0.125	$V_S(2)$ = 0.25*0.85*$V_V(1)$ = $3.5*10^{-2}$	$V_S(3)$ = 0.25*0.95*$V_S(2)$ = $8.3*10^{-3}$		

Iterative procedure:

- $V_k(i) = e_k(x_i) \times \max_j a_{jk} V_j(i-1)$

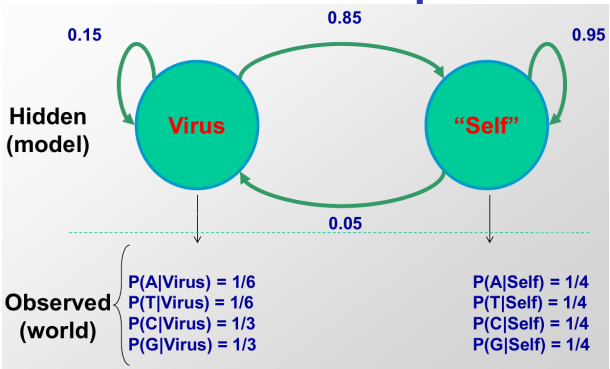
Step i=2: fill column 2

- $V_v(2) = e_V('C') \times \max\left(a_{VV} * V_V(1), a_{SV} * V_S(1)\right)$
  $= 0.33 \times \max\left(0.15 * 0.165, 0.05 * 0.125\right) = 8.17 * 10^{-3}$ (max: V at i=1!)
- $V_S(2) = e_S('C') \times \max\left(a_{VS} * V_V(1), a_{SS} * V_S(1)\right)$
  $= 0.25 \times \max\left(0.85 * 0.165, 0.95 * 0.125\right) = 3.5 * 10^{-2}$ (max: V at i=1!)

Step i=3: fill column 3

- $V_v(3) = e_V('G') \times \max\left(a_{VV} * V_V(2), a_{SV} * V_S(2)\right)$
  $= 0.33 \times \max\left(0.15 * 8.17 * 10^{-3}, 0.05 * 3.5 * 10^{-2}\right) = 5.8 * 10^{-4}$ (max: S at i=2!)
- $V_S(3) = e_S('G') \times \max\left(a_{VS} * V_V(2), a_{SS} * V_S(2)\right)$
  $= 0.25 \times \max\left(0.85 * 8.17 * 10^{-3}, 0.95 * 3.5 * 10^{-2}\right) = 8.3 * 10^{-3}$ (max: S at i=2!)

Viterbi: example



Step	1	2	3	4	5
Emission	G	C	G	A	T
State V=Virus	$V_V(1)$ = 0.5*0.33 = 0.165	$V_V(2)$ = 0.33*0.15*$V_V(1)$ = $8.1*10^{-3}$	$V_V(3)$ = 0.33*0.05*$V_S(2)$ = $5.8*10^{-4}$	$V_V(4)$ = 0.17*0.05*$V_S(3)$ = $7.1*10^{-5}$	$V_V(5)$ = 0.17*0.05*$V_S(4)$ = $1.7*10^{-5}$
State S="Self"	$V_S(1)$ = 0.5*0.25 = 0.125	$V_S(2)$ = 0.25*0.85*$V_V(1)$ = $3.5*10^{-2}$	$V_S(3)$ = 0.25*0.95*$V_S(2)$ = $8.3*10^{-3}$	$V_S(4)$ = 0.25*0.95*$V_S(3)$ = $2.0*10^{-3}$	**$V_S(5)$ = 0.25*0.95*$V_S(4)$ = $4.8*10^{-4}$**

Iterative procedure:

- $V_k(i) = e_k(x_i) \times \max_j a_{jk} V_j(i-1)$

Step i=4: fill column 4

- $V_v(4) = e_V('A') \times \max\left(a_{VV} * V_V(3), a_{SV} * V_S(3)\right)$
  $= 0.17 \times \max\left(0.15 * 5.8 * 10^{-4}, 0.05 * 8.3 * 10^{-3}\right) = 7.1 * 10^{-5}$ (max: S at i=3!)
- $V_S(4) = e_S('A') \times \max\left(a_{VS} * V_V(3), a_{SS} * V_S(3)\right)$
  $= 0.25 \times \max\left(0.85 * 5.8 * 10^{-4}, 0.95 * 8.3 * 10^{-3}\right) = 2.0 * 10^{-3}$ (max: S at i=3!)

Step i=5: fill column 5

- $V_v(5) = e_V('T') \times \max\left(a_{VV} * V_V(4), a_{SV} * V_S(4)\right)$
  $= 0.17 \times \max\left(0.15 * 7.1 * 10^{-5}, 0.05 * 2.0 * 10^{-3}\right) = 1.7 * 10^{-5}$ (max: S at i=4!)
- $V_S(5) = e_S('T') \times \max\left(a_{VS} * V_V(4), a_{SS} * V_S(4)\right)$
  $= 0.25 \times \max\left(0.85 * 7.1 * 10^{-5}, 0.95 * 2.0 * 10^{-3}\right) = 4.8 * 10^{-4}$ (max: S at i=4!)

Viterbi: example

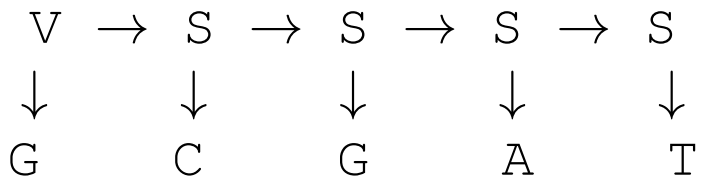
Step	1	2	3	4	5
Emission	G	C	G	A	T
State V=Virus	$V_V(1)$ = 0.5*0.33 = 0.165	$V_V(2)$ = 0.33*0.15*$V_V(1)$ = $8.1*10^{-3}$	$V_V(3)$ = 0.33*0.05*$V_S(2)$ = $5.8*10^{-4}$	$V_V(4)$ = 0.17*0.05*$V_S(3)$ = $7.1*10^{-5}$	$V_V(5)$ = 0.17*0.05*$V_S(4)$ = $1.7*10^{-5}$
State S="Self"	$V_S(1)$ = 0.5*0.25 = 0.125	$V_S(2)$ = 0.25*0.85*$V_V(1)$ = $3.5*10^{-2}$	$V_S(3)$ = 0.25*0.95*$V_S(2)$ = $8.3*10^{-3}$	$V_S(4)$ = 0.25*0.95*$V_S(3)$ = $2.0*10^{-3}$	**$V_S(5)$ = 0.25*0.95*$V_S(4)$ = $4.8*10^{-4}$**

The highest probability possible (= most likely path) at the last step (last column) is $4.8 * 10^{-4}$!

Traceback:

- Step 5: S
- Step 4: S
- Step 3: S
- Step 2: S
- Step 1: V

Most likely path:



The six algorithmic settings for HMMs

	One path	All paths
Scoring	1. Scoring x, one path $P(x,\pi)$ Prob of a path, emissions ✓	2. Scoring x, all paths $P(x) = \sum_\pi P(x,\pi)$ ★ Prob of emissions, over all paths
Decoding	3. Viterbi decoding $\pi^* = \text{argmax}_\pi P(x,\pi)$ Most likely path ✓	4. Posterior decoding $\pi\hat{} = \{\pi_i \mid \pi_i = \text{argmax}_k \sum_\pi P(\pi_i=k\mid x)\}$ Path containing the most likely state at any time point.
Learning	5. Supervised learning, given π $\Lambda^* = \text{argmax}_\Lambda P(x,\pi\mid\Lambda)$ 6. Unsupervised learning. $\Lambda^* = \text{argmax}_\Lambda \max_\pi P(x,\pi\mid\Lambda)$ Viterbi training, best path	6. Unsupervised learning $\Lambda^* = \text{argmax}_\Lambda \sum_\pi P(x,\pi\mid\Lambda)$ Baum-Welch training, over all paths

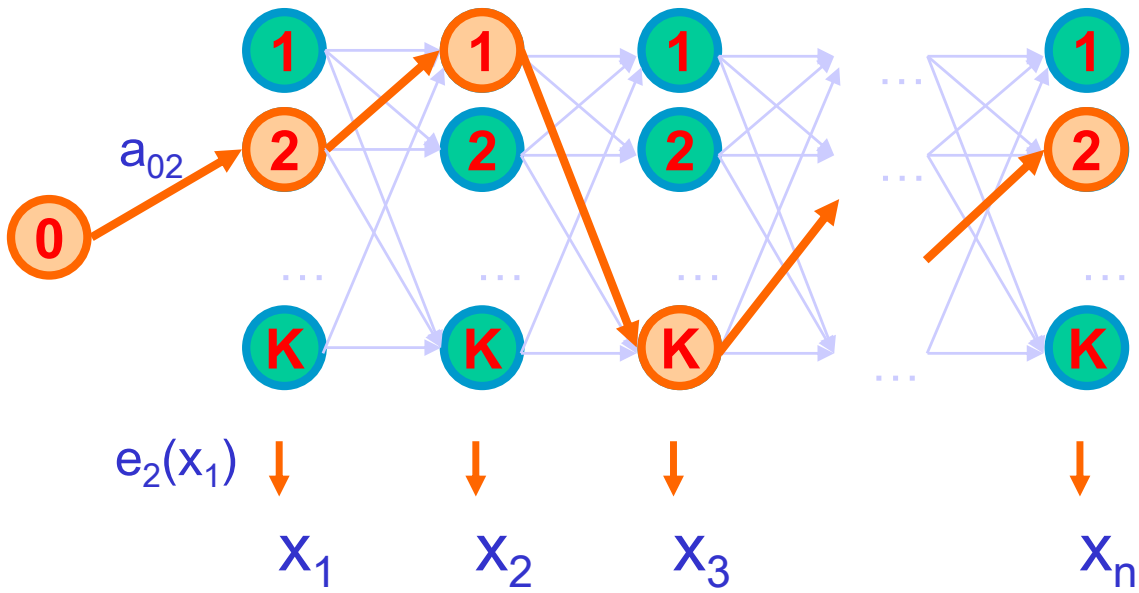
2. EVALUATION
(how well does our model capture the world)

Given: Model parameters $e_i(.)$, a_{ij}

Given: Sequence of emissions x

Find: P(x|M), summed over all possible paths π

Simple: Given the model, generate some sequence x



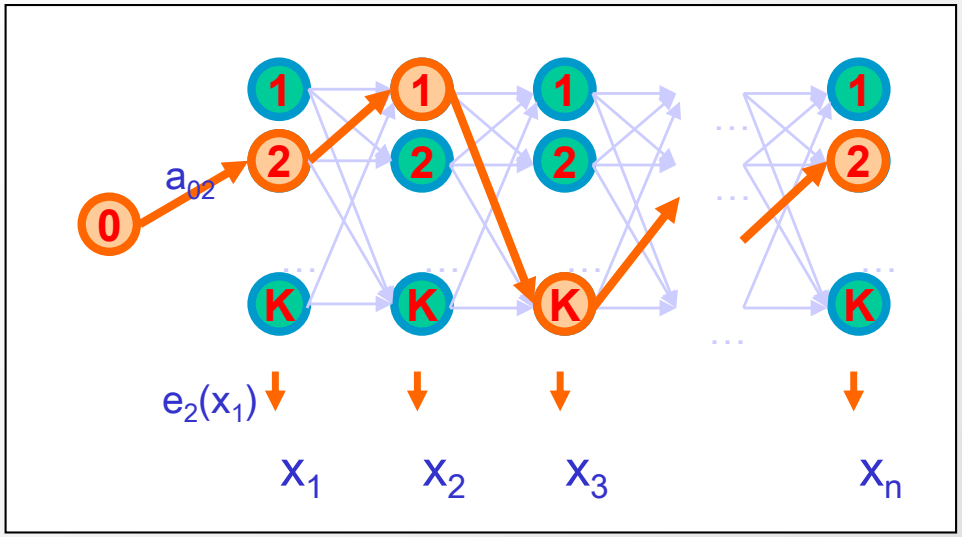
Given a HMM, we can generate a sequence of length n as follows:

1. Start at state π_1 according to prob $a_{0\pi1}$
2. Emit letter x_1 according to prob $e_{\pi1}(x_1)$
3. Go to state π_2 according to prob $a_{\pi1\pi2}$
4. … until emitting x_n

We have some sequence x that can be emitted by p. Can calculate its likelihood.
However, in general, many different paths may emit this same sequence x.
How do we find the total probability of generating a given x, over any path?

Slide credit: Serafim Batzoglou

Complex: Given x, was it generated by the model?



Given a sequence x,

What is the probability that x was generated by the model (using any path)?

- $P(x) = \sum_{\pi} P(x,\pi) = \sum_{\pi} P(x|\pi)\, P(\pi)$
- (weighted average of conditional probability, summed over all paths, weighted by each path's probability)

- Challenge: exponential number of paths

Slide credit: Serafim Batzoglou

Calculate probability of emission over all paths

- Each path has associated probability
  - Some paths are likely, others unlikely: sum them all up

  → Return total probability that emissions are observed, summed over all paths
  - Viterbi path is the most likely one
    - How much 'probability mass' does it contain?
- (cheap) alternative:
  - Calculate probability over maximum (Viterbi) path π^*
  - Good approximation if Viterbi has highest density
  - BUT: incorrect
- (real) solution
  - Calculate the exact sum iteratively
    - $P(x) = \sum_{\pi} P(x,\pi)$
  - Can use dynamic programming

Slide credit: Serafim Batzoglou

The Forward Algorithm – derivation

Define the forward probability:

$$f_l(i) = P(x_1 \ldots x_i, \pi_i = l)$$

$$= \sum_{\pi 1 \ldots \pi i-1} P(x_1 \ldots x_{i-1}, \pi_1, \ldots, \pi_{i-2}, \pi_{i-1}, \pi_i = l)\, e_l(x_i)$$

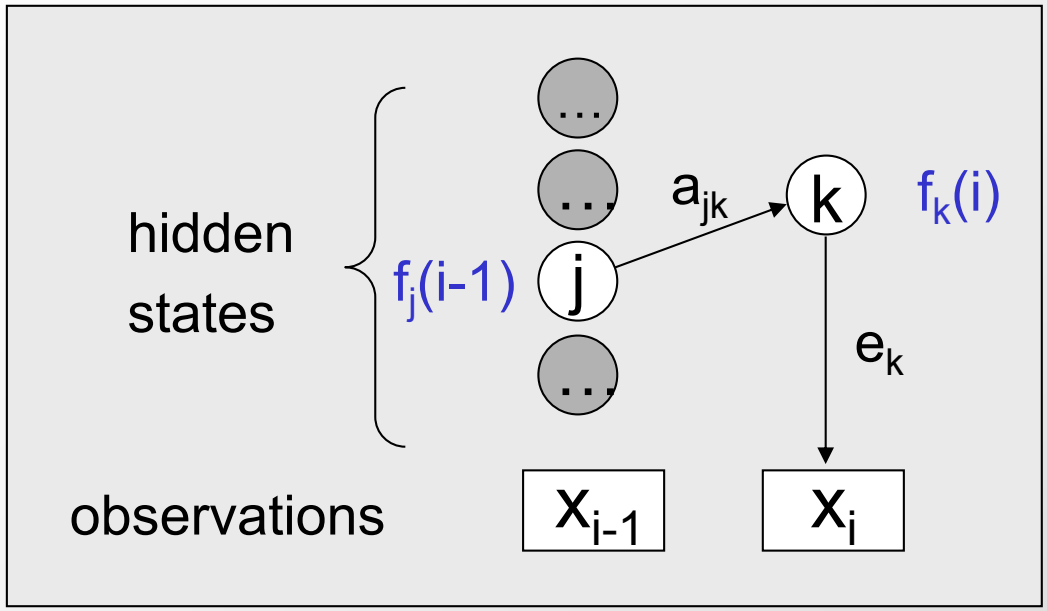
$$= \sum_k \boxed{\sum_{\pi 1 \ldots \pi i-2} P(x_1 \ldots x_{i-1}, \pi_1, \ldots, \pi_{i-2}, \pi_{i-1} = k)}\; a_{kl}\, e_l(x_i)$$

$$= \sum_k \boxed{f_k(i-1)}\, a_{kl}\, e_l(x_i)$$

$$= e_l(x_i) \sum_k \boxed{f_k(i-1)}\, a_{kl}$$

Slide credit: Serafim Batzoglou

Calculate total probability $\Sigma_\pi P(x,\pi)$ recursively



- Assume we know f_j for the previous time step (i-1)
- Calculate $f_k(i) = e_k(x_i) * sum_j (f_j(i-1) \times a_{jk})$

updated sum

this emission

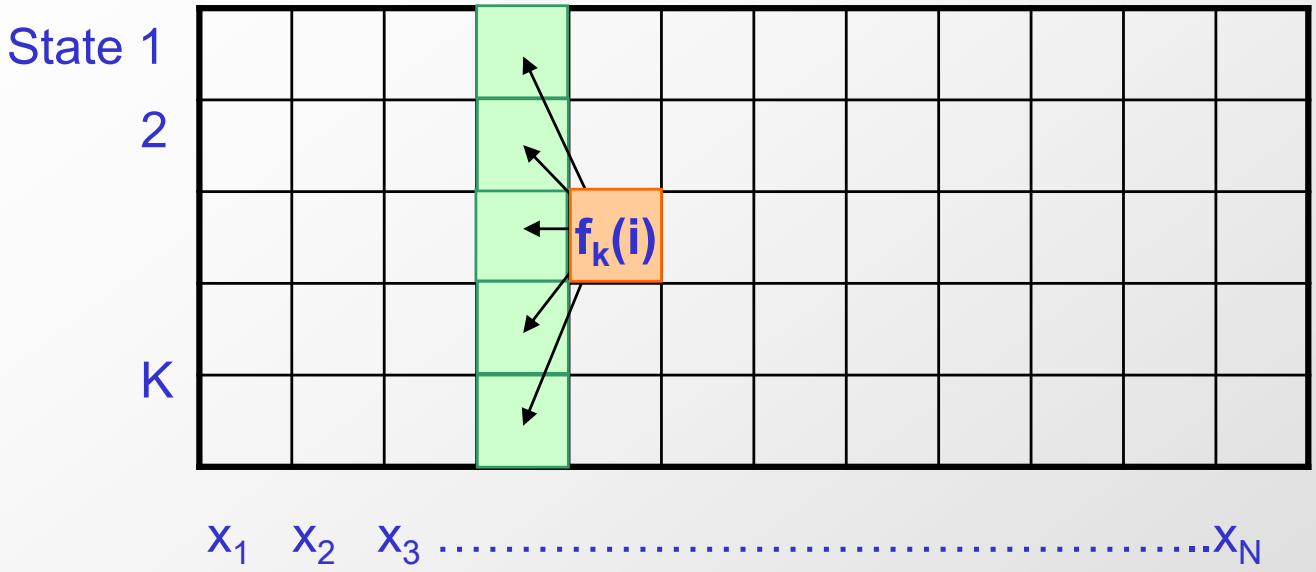
sum ending in state j at step i

transition from state j

every possible previous state j

Slide credit: Serafim Batzoglou

The Forward Algorithm



Input: x = x1……xN

Initialization:

$f_0(0)=1$, $f_k(0) = 0$, for all $k > 0$

Iteration:

$f_k(i) = e_K(x_i) \times \text{sum}_j\ a_{jk}\ f_j(i\text{-}1)$

Termination:

$P(x, \pi^*) = \text{sum}_k\ f_k(N)$

In practice:

Sum of log scores is difficult

→ approximate exp(1+p+q)

→ scaling of probabilities

Running time and space:

Time: $O(K^2N)$

Space: $O(KN)$

Slide credit: Serafim Batzoglou

The six algorithmic settings for HMMs

	One path	All paths
Scoring	1. Scoring x, one path $P(x,\pi)$ Prob of a path, emissions ✓	2. Scoring x, all paths $P(x) = \sum_\pi P(x,\pi)$ Prob of emissions, over all paths ✓
Decoding	3. Viterbi decoding $\pi^* = \text{argmax}_\pi P(x,\pi)$ Most likely path ✓	4. Posterior decoding $\pi\hat{} = \{\pi_i \mid \pi_i = \text{argmax}_k \sum_\pi P(\pi_i=k\mid x)\}$ Path containing the most likely state at any time point.
Learning	5. Supervised learning, given π $\Lambda^* = \text{argmax}_\Lambda P(x,\pi\mid\Lambda)$ 6. Unsupervised learning. $\Lambda^* = \text{argmax}_\Lambda \max_\pi P(x,\pi\mid\Lambda)$ Viterbi training, best path	6. Unsupervised learning $\Lambda^* = \text{argmax}_\Lambda \sum_\pi P(x,\pi\mid\Lambda)$ Baum-Welch training, over all paths

Increasing the state space (remembering more)

HMM1: Promoters = **only Cs and Gs matter**

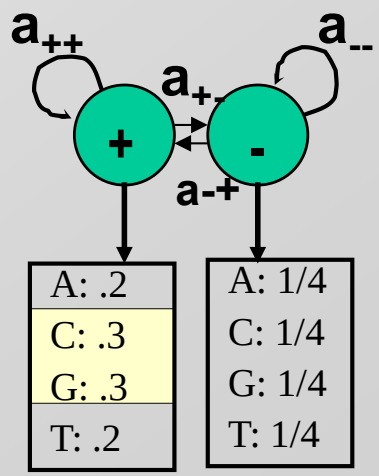
HMM2: Promoters = **it's actually CpGs that matter**

(di-nucleotides, remember previous nucleotide)

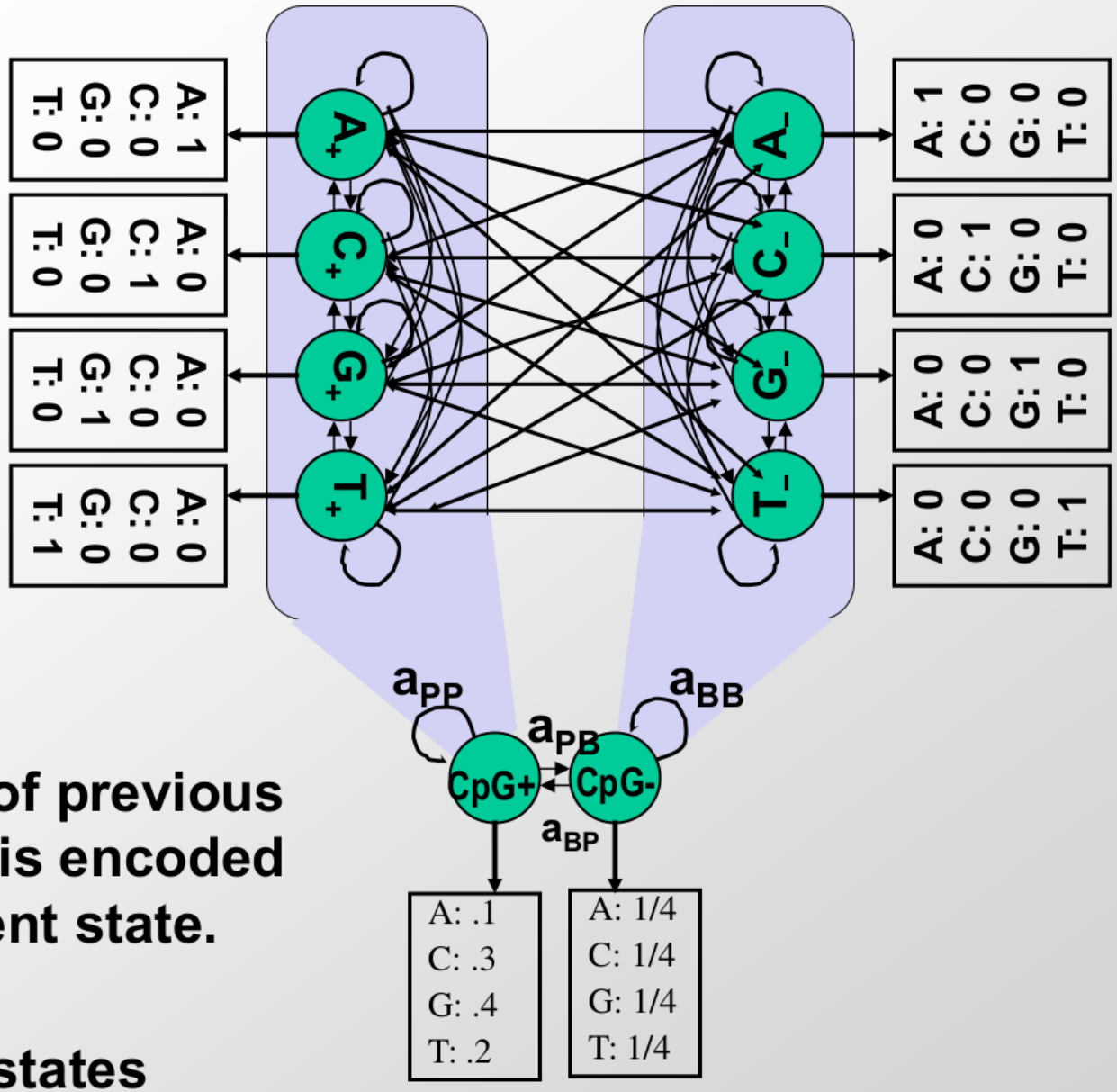
Increasing the state of the system (looking back)

- Markov Models are memory-less
  - In other words, all memory is encoded in the states
  - To remember additional information, augment state
- A two-state HMM has minimal memory
  - Two states: GC-rich vs. equal probability
  - State, emissions, only depend on **current** state
  - Current state only encodes **one** previous nucleotide
- How do you count **di**-nucleotide frequencies?
  - CpG islands: di-nucleotides
  - Codon triplets: tri-nucleotides
  - Di-codon frequencies: six nucleotides

➔ Expanding the number of states



Remember previous nucleotide: expand both states

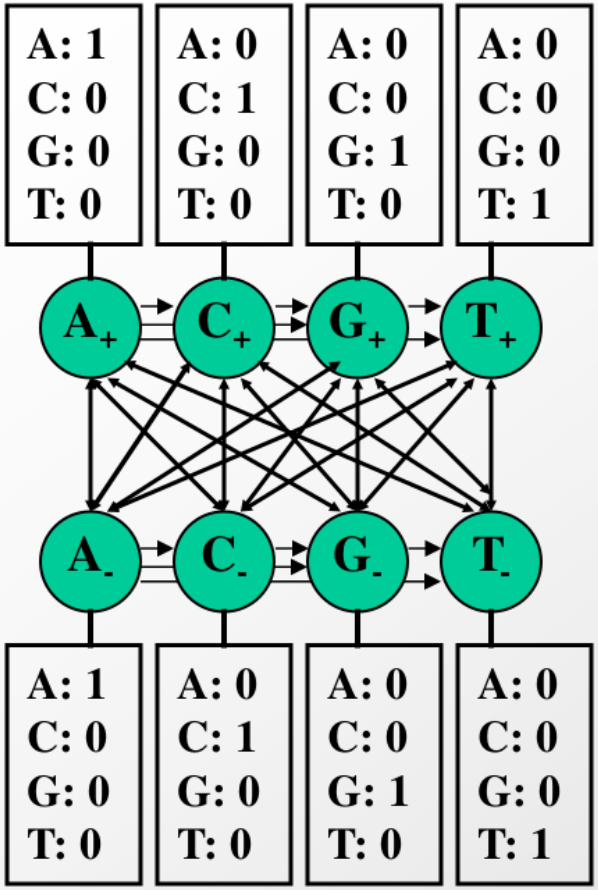


"Memory" of previous nucleotide is encoded in the current state.

GC-rich: 4 states

Background: 4 states

HMM for CpG islands



- A single model combines two Markov chains, each of four nucleotides:
  - **'+' states**: A_+, C_+, G_+, T_+
    - Emit symbols: A, C, G, T in CpG islands
  - **'-' states**: A_-, C_-, G_-, T_-
    - Emit symbols: A, C, G, T in non-islands
- Trans-mission probabilities distinct for the '+' and the '-' states
  - Infer most likely set of states, giving rise to observed emissions

➔ 'Paint' the sequence with + and - states

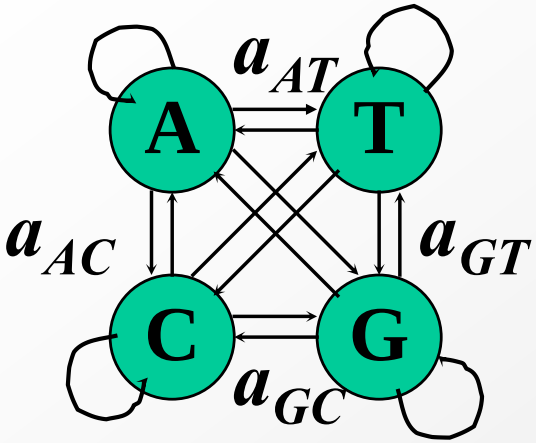
Why we need so many states…

In our simple GC-content example, we only had 2 states (+|-)

Why do we need 8 states here: 4 CpG+ / 4 CpG- ?

➔ Encode 'memory' of previous state: nucleotide transitions

Training emission parameters for CpG+/CpG- states



- Count di-nucleotide frequencies:
  - 16 possible di-nucleotides. 16 transition parameters.
  - Alternative: 16 states, each emitting di-nucleotide
- Derive two Markov chain models:
  - **'+' model**: from the CpG islands
  - **'-' model**: from the remainder of sequence
- Transition probabilities for each model:
  - Encode differences in di-nucleotide frequencies

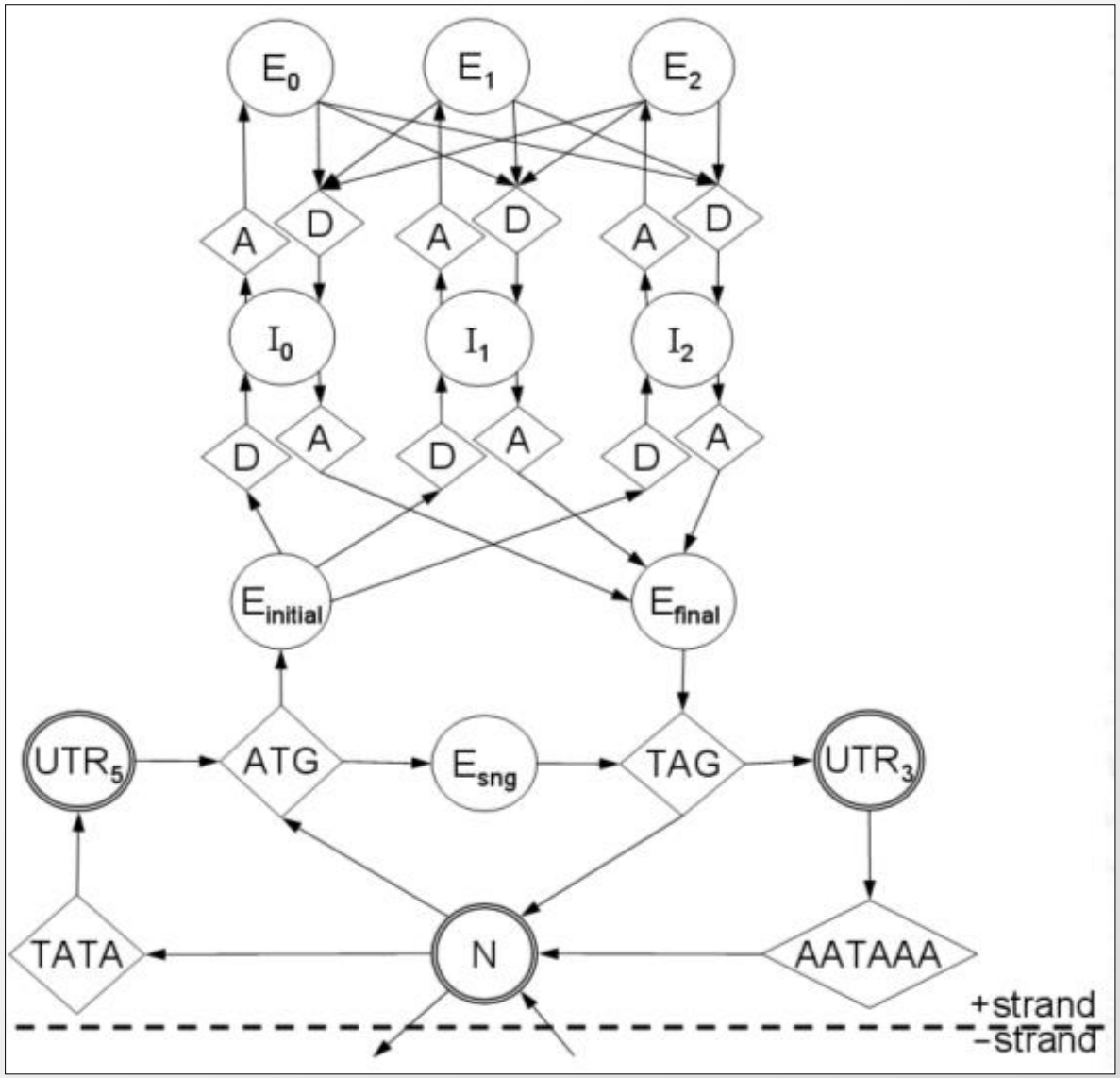
+	A	C	G	T
A	.180	.274	.426	.120
C	.171	.368	**.274**	.188
G	.161	.339	.375	.125
T	.079	.355	.384	.182

-	A	C	G	T
A	.300	.205	.285	.210
C	.322	.298	**.078**	.302
G	.248	.246	.298	.208
T	.177	.239	.292	.292

Examples of HMMs for genome annotation

Detection of GC-rich regions	**Detection of CpG-rich regions**	**Detection of conserved regions**	**Detection of protein-coding exons**	**Detection of protein-coding conservation**	**Detection of protein-coding gene structures**	**Detection of chromatin states**
2 states, different nucleotide composition	8 states, 4 each +/-, different transition probabilities	2 states, different conservation levels	2 states, different tri-nucleotide composition	2 states, different evolutionary signatures	~20 states, different composition/ conservation, specific structure	40 states, different chromatin mark combinations
GC-rich / AT-rich	CpG-rich / CpG-poor	Conserved / non-conserved	Coding exon / non-coding (intron or intergenic)	Coding exon / non-coding (intron or intergenic)	First/last/middle coding exon,UTRs, intron1/2/3, intergenic, *(+/- strand)	Enhancer / promoter / transcribed / repressed / repetitive
Nucleotides	Di-Nucleotides	Level of conservation	Triplets of nucleotides	64x64 matrix of codon substitution frequencies	Codons, nucleotides, splice sites, start/stop codons	Vector of chromatin mark frequencies

HMM architecture matters: Protein-coding genes



© Bill Majoros / GeneZilla. All rights reserved. This content is excluded from our Creative Commons license. For more information, see http://ocw.mit.edu/help/faq-fair-use/.

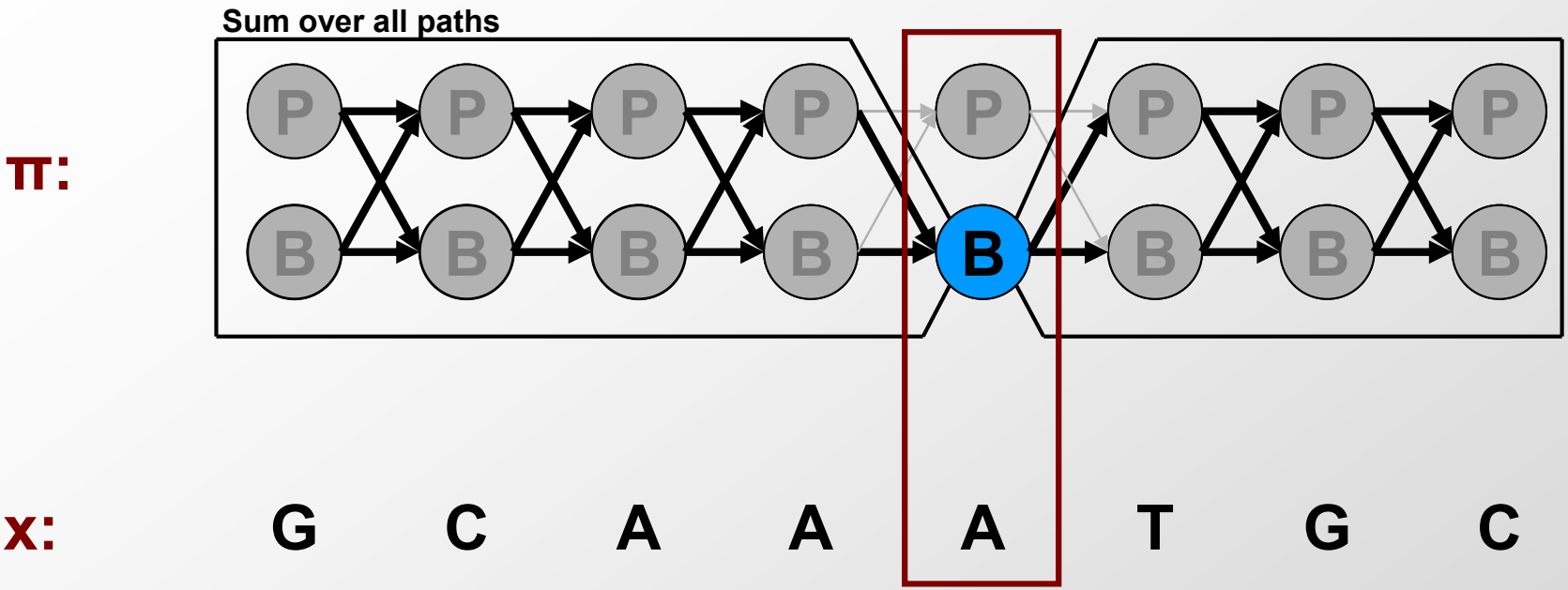
- Gene vs. Intergenic
- Start & Stop in/out
- UTR: 5' and 3' end
- Exons, Introns
- Remembering frame
  - E0,E1,E2
  - I0,I1,I2
- Sequence patterns to transition between states:
  - ATG, TAG, Acceptor/Donor, TATA, AATAA

	One path	All paths
Scoring	1. Scoring x, one path $P(x,\pi)$ Prob of a path, emissions ✓	2. Scoring x, all paths $P(x) = \sum_\pi P(x,\pi)$ ✓ Prob of emissions, over all paths
Decoding	3. Viterbi decoding $\pi^* = \text{argmax}_\pi P(x,\pi)$ Most likely path ✓	4. Posterior decoding $\pi\text{^} = \{\pi_i \mid \pi_i = \text{argmax}_k \sum_\pi P(\pi_i = k \mid x)\}$ Path containing the most likely state at any time point.
Learning	5. Supervised learning, given π $\Lambda^* = \text{argmax}_\Lambda P(x,\pi\mid\Lambda)$ 6. Unsupervised learning. $\Lambda^* = \text{argmax}_\Lambda \max_\pi P(x,\pi\mid\Lambda)$ Viterbi training, best path	6. Unsupervised learning $\Lambda^* = \text{argmax}_\Lambda \sum_\pi P(x,\pi\mid\Lambda)$ Baum-Welch training, over all paths

4. Decoding, all paths

Find the likelihood an emission x_i is generated by a state

Calculate most probable label at a single position



$P(Label_i=B|x)$

- Calculate most probable label, L^*_i , at each position i
- Do this for all N positions gives us $\{L^*_1, L^*_2, L^*_3 \ldots L^*_N\}$
- How much information have we observed? Three settings:
  - **Observed nothing: Use prior information**
  - **Observed only character at position i: Prior + emission probability**
  - **Observed entire sequence: Posterior decoding**

Calculate $P(\pi_7 = \text{CpG+} \mid x_7 = G)$

- With no knowledge (no characters)
  - Simply time spent in markov chain states
  - $P(\pi_i = k)$ = most likely state (**prior**)

- With very little knowledge (just that character)
  - Time spent, adjusted for different emission probs.
  - Use Bayes rule to change inference directionality
  - $P(\pi_i = k \mid x_i = G) = P(\pi_i = \kappa) * P(x_i = G \mid \pi_i = k) / P(x_i = G)$

- With knowledge of entire sequence (all characters)
  - $P(\pi_i = k \mid x = \text{AGCGCG…GATTATCGTCGTA})$
  - Sum over all paths that emit 'G' at position 7

  ➔ **Posterior** decoding

Motivation for the Backward Algorithm

We want to compute

$P(\pi_i = k \mid x)$, the probability distribution on the ith position, given x

We start by computing

$$P(\pi_i = k, x) = P(x_1 \ldots x_i, \pi_i = k, x_{i+1} \ldots x_N)$$
$$= P(x_1 \ldots x_i, \pi_i = k)\, P(x_{i+1} \ldots x_N \mid x_1 \ldots x_i, \pi_i = k)$$
$$= P(x_1 \ldots x_i, \pi_i = k)\, P(x_{i+1} \ldots x_N \mid \pi_i = k)$$

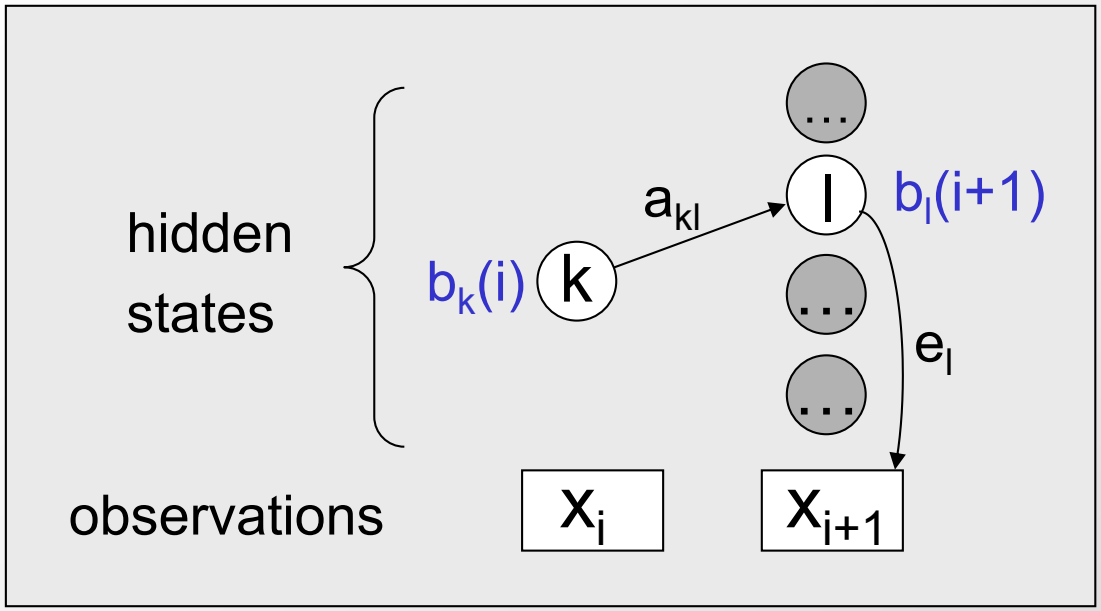
Forward, $f_k(i)$ Backward, $b_k(i)$

The Backward Algorithm – derivation

Define the backward probability:

$$
\begin{aligned}
b_k(i) &= P(x_{i+1} \ldots x_N \mid \pi_i = k) \\
&= \Sigma_{\pi i+1 \ldots \pi N} P(x_{i+1}, x_{i+2}, \ldots, x_N, \pi_{i+1}, \ldots, \pi_N \mid \pi_i = k) \\
&= \Sigma_l \Sigma_{\pi i+2 \ldots \pi N} P(x_{i+1}, x_{i+2}, \ldots, x_N, \pi_{i+1} = l, \pi_{i+2}, \ldots, \pi_N \mid \pi_i = k) \\
&= \Sigma_l e_l(x_{i+1}) a_{kl} \boxed{\Sigma_{\pi i+2 \ldots \pi N} P(x_{i+2}, \ldots, x_N, \pi_{i+2}, \ldots, \pi_N \mid \pi_{i+1} = l)} \\
&= \Sigma_l e_l(x_{i+1}) a_{kl} \boxed{b_l(i+1)}
\end{aligned}
$$

Calculate total end probability recursively



- Assume we know b_l for the next time step (i+1)
- Calculate $b_k(i)$ = sum_l ($e_l(x_{i+1})$ × a_{kl} × $b_l(i+1)$)

$b_k(i)$: current max

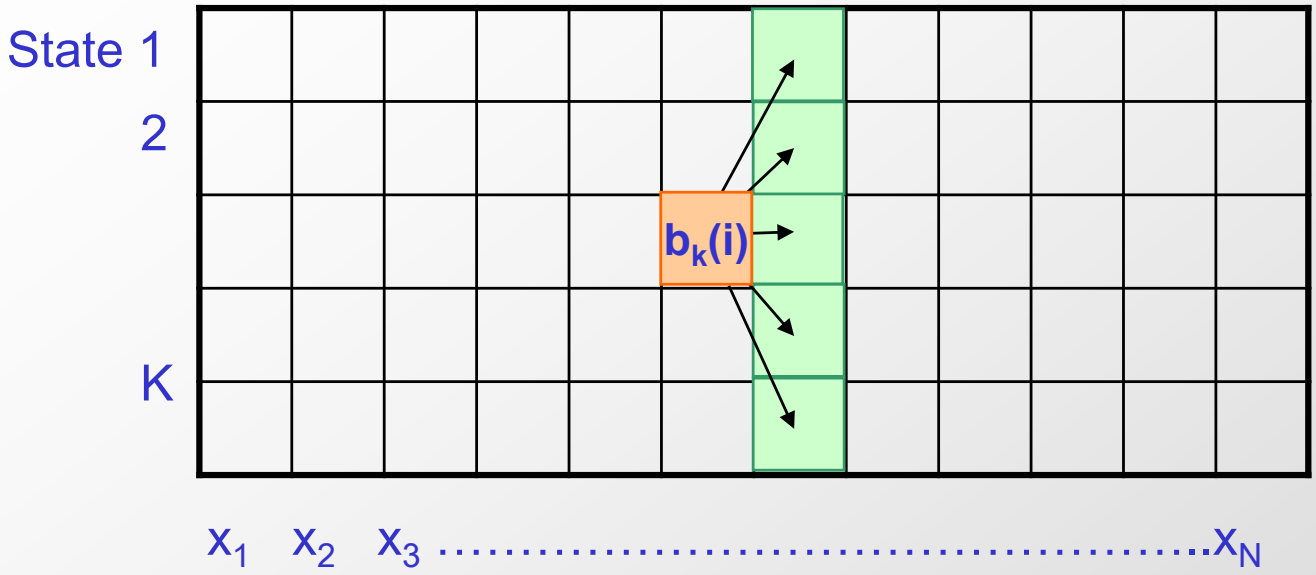
$e_l(x_{i+1})$: next emission

a_{kl}: transition to next state

$b_l(i+1)$: prob sum from state l to end

sum_l: sum over all possible next states

The Backward Algorithm



Input: x = x1……xN

Initialization:

$b_k(N) = a_{k0}$, for all k

Iteration:

$b_k(i) = \Sigma_l\, e_l(x_{i+1})\, a_{kl}\, b_l(i+1)$

Termination:

$P(x) = \Sigma_l\, a_{0l}\, e_l(x_1)\, b_l(1)$

In practice:

Sum of log scores is difficult

→ approximate exp(1+p+q)

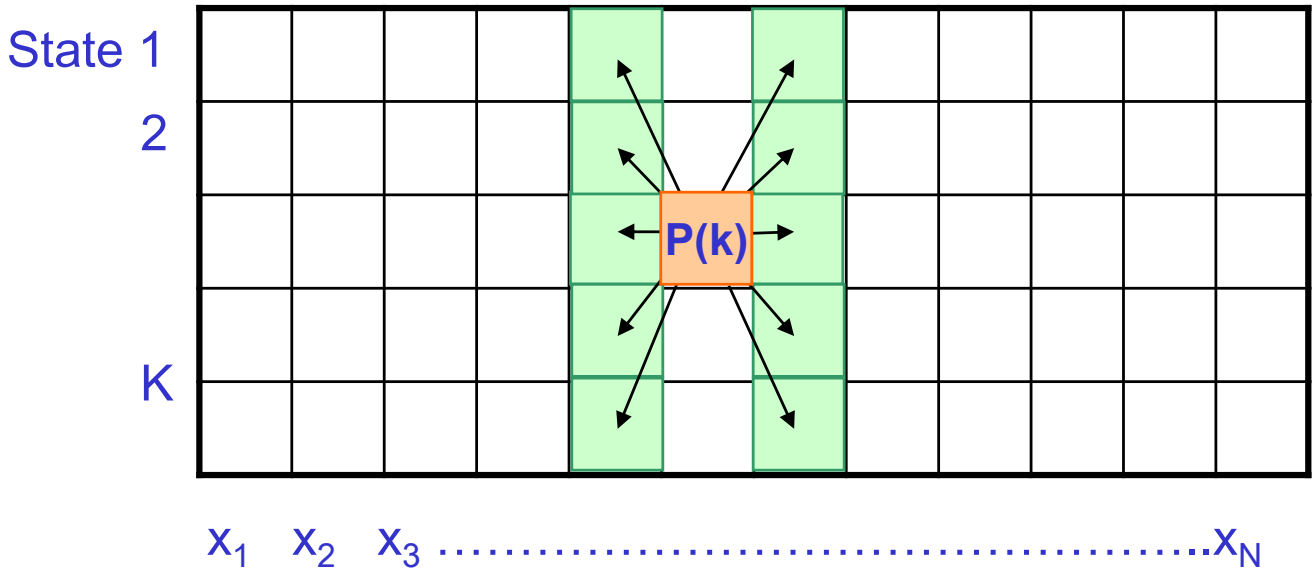
→ scaling of probabilities

Running time and space:

Time: $O(K^2N)$

Space: $O(K)$

Putting it all together: Posterior decoding



- $P(k) = P(\pi_i=k \mid x) = f_k(i)*b_k(i) / P(x)$
  - Probability that i^{th} state is k, given all emissions x
- Posterior decoding
  - Find the most likely state at position i over all possible hidden paths given the observed sequence x
  - $\hat{\pi}_i = \text{argmax}_k P(\pi_i = k \mid x)$
- Posterior decoding 'path' $\hat{\pi}_i$
  - For classification, more informative than Viterbi path π^*
    - More refined measure of "which hidden states" generated x
  - However, it may give an invalid sequence of states
    - Not all j→k transitions may be possible

	One path	All paths
Scoring	1. Scoring x, one path $P(x,\pi)$ Prob of a path, emissions	2. Scoring x, all paths $P(x) = \sum_{\pi} P(x,\pi)$ Prob of emissions, over all paths
Decoding	3. Viterbi decoding $\pi^* = \text{argmax}_{\pi} P(x,\pi)$ Most likely path	4. Posterior decoding $\pi\text{^} = \{\pi_i \mid \pi_i = \text{argmax}_k \sum_{\pi} P(\pi_i = k \mid x)\}$ Path containing the most likely state at any time point.
Learning	5. Supervised learning, given π $\Lambda^* = \text{argmax}_{\Lambda} P(x,\pi\mid\Lambda)$ 6. Unsupervised learning. $\Lambda^* = \text{argmax}_{\Lambda} \max_{\pi} P(x,\pi\mid\Lambda)$ Viterbi training, best path	6. Unsupervised learning $\Lambda^* = \text{argmax}_{\Lambda} \sum_{\pi} P(x,\pi\mid\Lambda)$ Baum-Welch training, over all paths

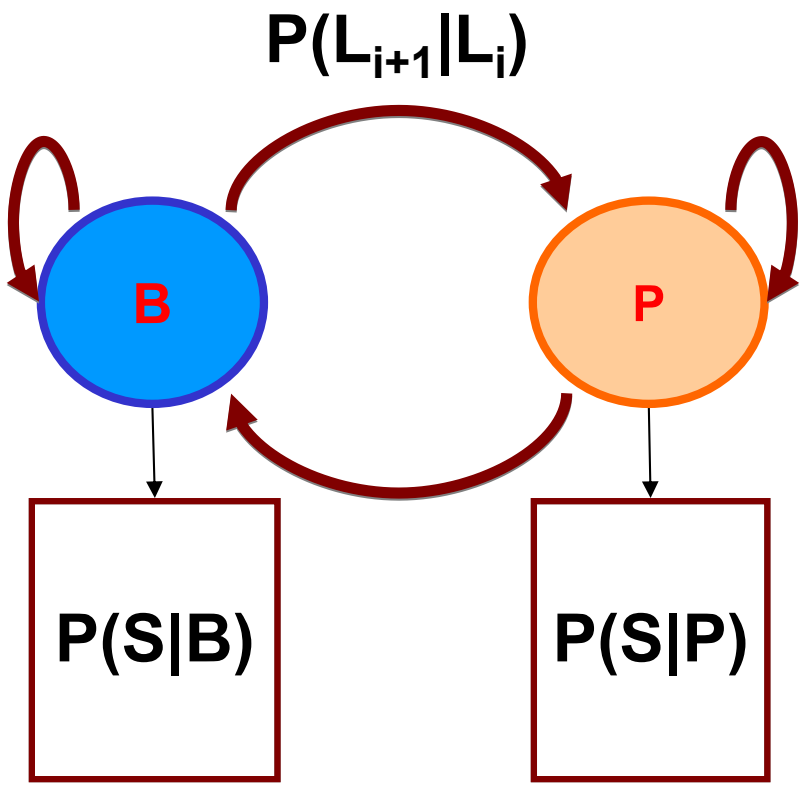
Learning: How to train an HMM

Transition probabilities

e.g. $P(P_{i+1}|B_i)$ – the probability of entering a pathogenicity island from background DNA

Emission probabilities

i.e. the nucleotide frequencies for background DNA and pathogenicity islands



Two learning scenarios

Case 1. Estimation when the "right answer" is known

Examples:

GIVEN: a genomic region $x = x_1 \ldots x_{1,000,000}$ where we have good (experimental) annotations of the CpG islands

Case 2. Estimation when the "right answer" is unknown

Examples:

GIVEN: the porcupine genome; we don't know how frequent are the CpG islands there, neither do we know their composition

QUESTION: Update the parameters θ of the model to maximize $P(x|\theta)$

Two types of learning: Supervised / Unsupervised

5. Supervised learning

infer model parameters given **labeled** training data

- GIVEN:
  - a HMM M, with unspecified transition/emission probs.
  - labeled sequence x,
- FIND:
  - parameters θ = (Ei, Aij) that maximize P[x | θ]

➔ Simply count frequency of each emission and transition, as observed in the training data

6. Unsupervised learning

infer model parameters given **unlabelled** training data

- GIVEN:
  - a HMM M, with unspecified transition/emission probs.
  - unlabeled sequence x,
- FIND:
  - parameters θ = (Ei, Aij) that maximize P[x | θ]

➔ Viterbi training:
guess parameters, find optimal Viterbi path (#2), update parameters (#5), iterate

➔ Baum-Welch training:
guess parameters, sum over all paths (#4), update parameters (#5), iterate

5: Supervised learning

Estimate model parameters
based on **labeled** training data

Case 1. When the right answer is known

Given $x = x_1 \ldots x_N$
for which the true $\pi = \pi_1 \ldots \pi_N$ is known,

Define:

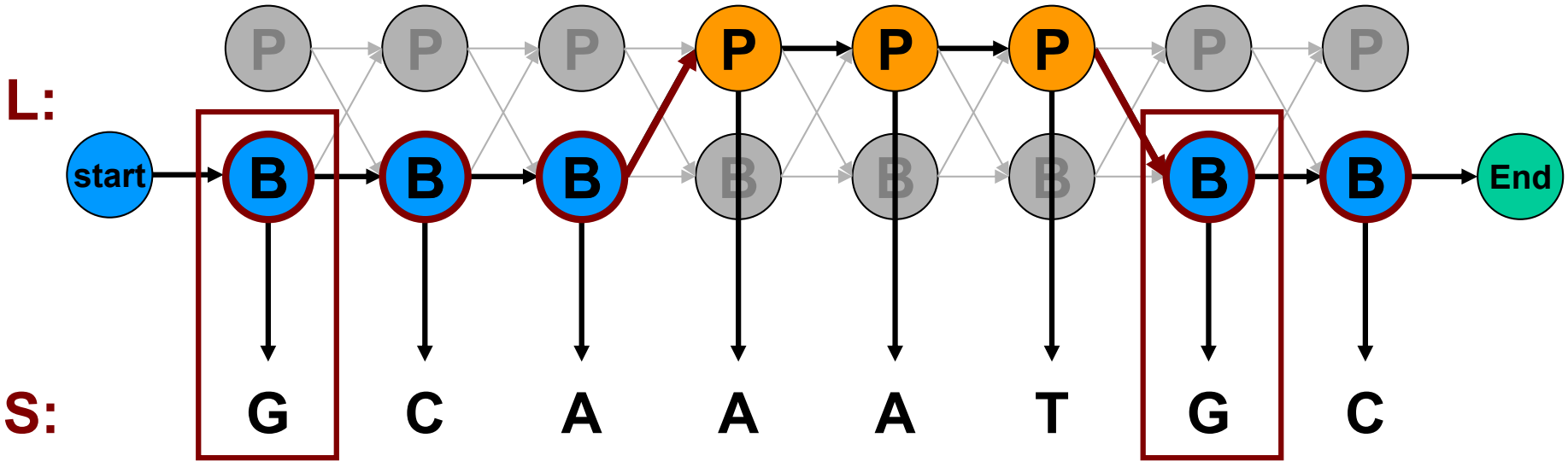
A_{kl} = # times $k \rightarrow l$ transition occurs in π
$E_k(b)$ = # times state k in π emits b in x

We can show that the maximum likelihood parameters θ are:

$$a_{kl} = \frac{A_{kl}}{\sum_i A_{ki}} \qquad e_k(b) = \frac{E_k(b)}{\sum_c E_k(c)}$$

Learning From Labelled Data
→ Maximum Likelihood Estimation

If we have a sequence that has islands marked, we can simply count



$P(L_{i+1}|L_i)$

	B_{i+1}	P_{i+1}
B_i		
P_i		

$P(S|B)$

A:
T:
G:
C:
!

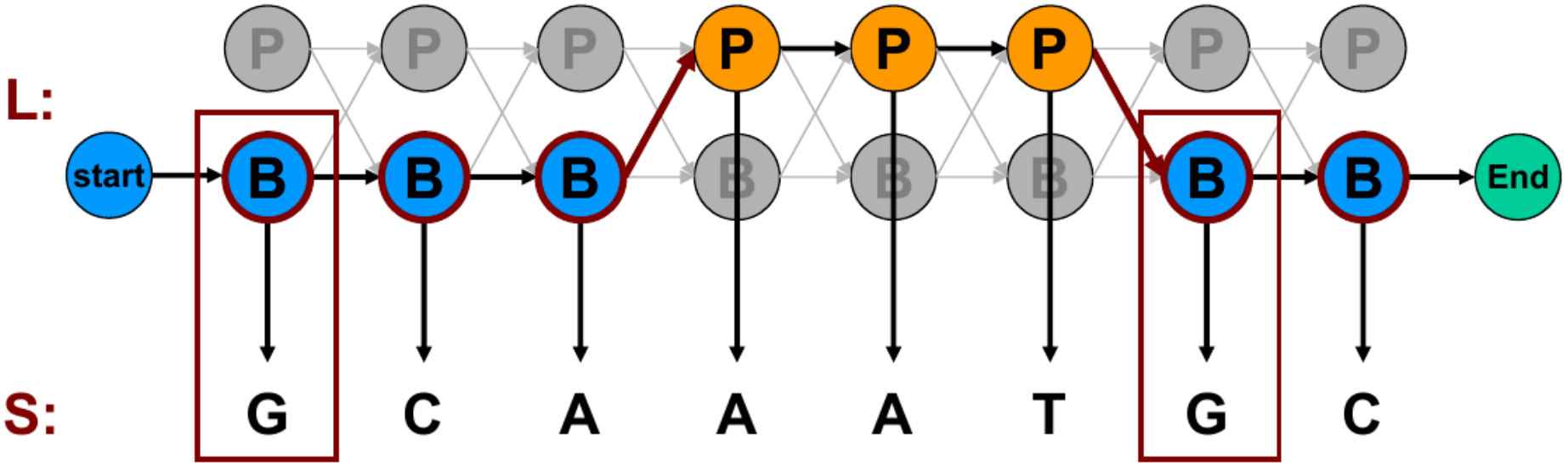
$P(S|P)$

A:
T:
G:
C:
ETC..

Learning From Labelled Data

→ Maximum Likelihood Estimation

If we have a sequence that has islands marked, we can simply count



$P(L_{i+1}|L_i)$

	B_{i+1}	P_{i+1}
B_i	3/4	1 / 4
P_i	1/3	2/3

$P(S|B)$

A: 1/5
T: 0/5
G: 2 / 5
C: 2/5
!

$P(S|P)$

A:
T:
G:
C:
ETC..

Case 1. When the right answer is known

Intuition: When we know the underlying states,
Best estimate is the average frequency of
transitions & emissions that occur in the training data

Drawback:

Given little data, there may be **overfitting**:
$P(x|\theta)$ is maximized, but θ is unreasonable
0 probabilities – VERY BAD

Example:

Given 10 nucleotides, we observe

x = C, A, G, G, T, C, C, A, T, C
π = P, P, P, p, p, P, P, P, P, P

Then:

$a_{PP} = 1$; $a_{PB} = 0$
$e_P(A) = .2$;
$e_P(C) = .4$;
$e_P(G) = .2$;
$e_P(T) = .2$

Pseudocounts

Solution for small training sets:

Add pseudocounts

A_{kl} = # times k$\rightarrow$l transition occurs in π + r_{kl}

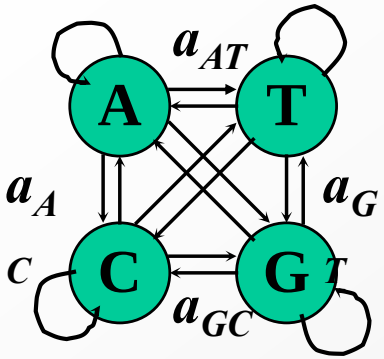
$E_k(b)$ = # times state k in π emits b in x + $r_k(b)$

r_{kl}, $r_k(b)$ are pseudocounts representing our prior belief

Larger pseudocounts $\Rightarrow$ Strong priof belief

Small pseudocounts (ε < 1): just to avoid 0 probabilities

Example: Training Markov Chains for CpG islands



- Training Set:
  - set of DNA sequences w/ known CpG islands
- Derive two Markov chain models:
  - **'+' model**: from the CpG islands
  - **'-' model**: from the remainder of sequence
- Transition probabilities for each model:

+	A	C	G	T
A	.180	.274	.426	.120
C	.171	.368	**.274**	.188
G	.161	.339	.375	.125
T	.079	.355	.384	.182

-	A	C	G	T
A	.300	.205	.285	.210
C	.322	.298	**.078**	.302
G	.248	.246	.298	.208
T	.177	.239	.292	.292

$$a_{st}^{+} = \frac{c_{st}^{+}}{\sum_{t'} c_{st'}^{+}}$$

c_{st}^{+} **is the number of times letter *t* followed letter *s* inside the CpG islands**

$$a_{st}^{-} = \frac{c_{st}^{-}}{\sum_{t'} c_{st'}^{-}}$$

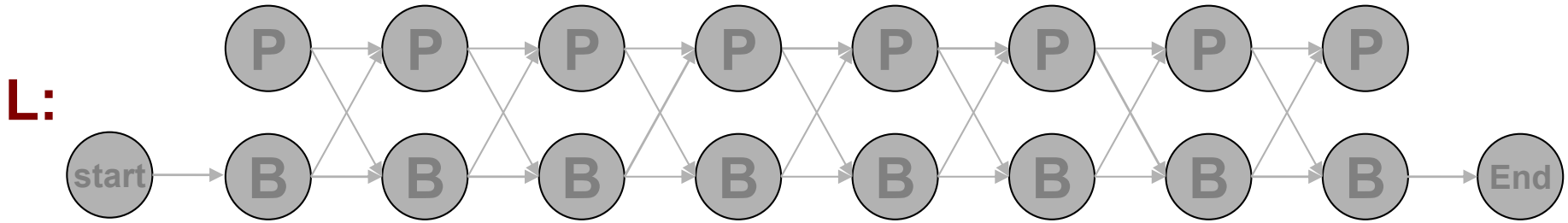
c_{st}^{-} **is the number of times letter *t* followed letter *s* outside the CpG islands**

6: Unsupervised learning

Estimate model parameters
based on **unlabeled** training data

Unlabelled Data

How do we know how to count?



S: G C A A A T G C

?

$P(L_{i+1}|L_i)$

	B_{i+1}	P_{i+1}	End
B_i			
P_i			
Start			

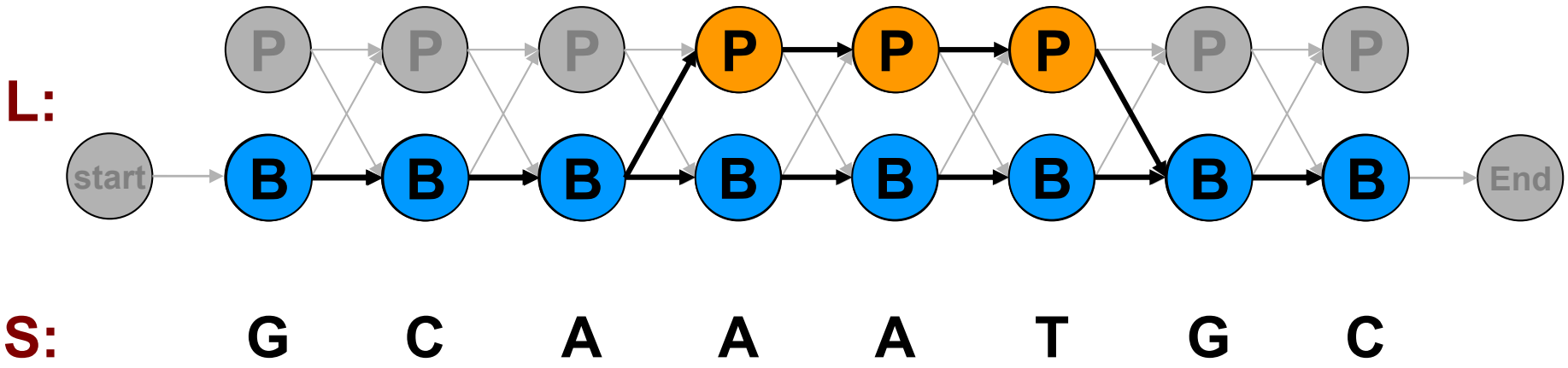
P(S|B)

A:
T:
G:
C:

P(S|P)

A:
T:
G:
C:

Unlabeled Data



An idea:

1. Imagine we start with some parameters
2. We *could* calculate the most likely path, P*, given those parameters and S
3. We *could* then use P* to update our parameters by maximum likelihood
4. And iterate (to convergence)

~~$P(L_{i+1}|L_i)^0$ $P(S|B)^0$ $P(S|P)^0$~~

~~$P(L_{i+1}|L_i)^1$ $P(S|B)^1$ $P(S|P)^1$~~

$P(L_{i+1}|L_i)^2$ $P(S|B)^2$ $P(S|P)^2$

…

$P(L_{i+1}|L_i)^K$ $P(S|B)^K$ $P(S|P)^K$

Learning case 2. When the right answer is unknown

We don't know the true A_{kl}, $E_k(b)$

Idea:

- We estimate our "best guess" on what A_{kl}, $E_k(b)$ are (M step, maximum-likelihood estimation)
- We update the probabilistic parse of our sequence, based on these parameters (E step, expected probability of being in each state given parameters)
- We repeat

Two settings:

- Simple: Viterbi training (best guest = best path)
- Correct: Expectation maximization (all paths, weighted)

	One path	All paths
Scoring	1. Scoring x, one path $P(x,\pi)$ Prob of a path, emissions ✓	2. Scoring x, all paths $P(x) = \sum_\pi P(x,\pi)$ Prob of emissions, over all paths ✓
Decoding	3. Viterbi decoding $\pi^* = \text{argmax}_\pi P(x,\pi)$ Most likely path ✓	4. Posterior decoding $\pi\text{^} = \{\pi_i \mid \pi_i = \text{argmax}_k \sum_\pi P(\pi_i = k \mid x)\}$ Path containing the most likely state at any time point. ✓
Learning	5. Supervised learning, given π $\Lambda^* = \text{argmax}_\Lambda P(x,\pi\mid\Lambda)$ ✓ 6. Unsupervised learning. $\Lambda^* = \text{argmax}_\Lambda \max_\pi P(x,\pi\mid\Lambda)$ Viterbi training, best path	7. Unsupervised learning $\Lambda^* = \text{argmax}_\Lambda \sum_\pi P(x,\pi\mid\Lambda)$ Baum-Welch training, over all paths

Simple casae: Viterbi Training

Initialization:

Pick the best-guess for model parameters

(or arbitrary)

Iteration:

1. Perform Viterbi, to find π^*
2. Calculate A_{kl}, $E_k(b)$ according to π^* + pseudocounts
3. Calculate the new parameters a_{kl}, $e_k(b)$

Until convergence

Notes:

- Convergence to local maximum guaranteed. Why?
- Does not maximize $P(x \mid \theta)$
- In general, worse performance than Baum-Welch

	One path	All paths
Scoring	1. Scoring x, one path $P(x,\pi)$ Prob of a path, emissions	2. Scoring x, all paths $P(x) = \sum_\pi P(x,\pi)$ Prob of emissions, over all paths
Decoding	3. Viterbi decoding $\pi^* = \text{argmax}_\pi P(x,\pi)$ Most likely path	4. Posterior decoding $\pi\text{^} = \{\pi_i \mid \pi_i = \text{argmax}_k \sum_\pi P(\pi_i = k \mid x)\}$ Path containing the most likely state at any time point.
Learning	5. Supervised learning, given π $\Lambda^* = \text{argmax}_\Lambda P(x,\pi\mid\Lambda)$ 6. Unsupervised learning. $\Lambda^* = \text{argmax}_\Lambda \max_\pi P(x,\pi\mid\Lambda)$ Viterbi training, best path	6. Unsupervised learning $\Lambda^* = \text{argmax}_\Lambda \sum_\pi P(x,\pi\mid\Lambda)$ Baum-Welch training, over all paths

Expectation Maximization (EM)

The basic idea is the same:

1.Use model to estimate missing data (E step)
2.Use estimate to update model (M step)
3.Repeat until convergence

EM is a general approach for learning models (ML estimation) when there is "missing data"
Widely used in computational biology

EM pervasive in computational biology

Expectation Maximization (EM)

1. **Initialize parameters randomly**

2. **E Step Estimate expected probability of hidden labels, Q, given current (latest) parameters and observed (unchanging) sequence**

$$Q = P(Labels | S, params^{t-1})$$

3. **M Step Choose new maximum likelihood parameters over probability distribution Q, given current probabilistic label assignments**

$$params^{t} = \arg\max_{params} E_{Q}\left[\log P(S, labels \mid params^{t-1})\right]$$

4. **Iterate**

P(S|Model) *guaranteed* to increase each iteration

Case 2. When the right answer is unknown

Starting with our best guess of a model M, parameters θ:

Given $x = x_1 \ldots x_N$

for which the true $\pi = \pi_1 \ldots \pi_N$ is unknown,

We can get to a provably more likely parameter set θ

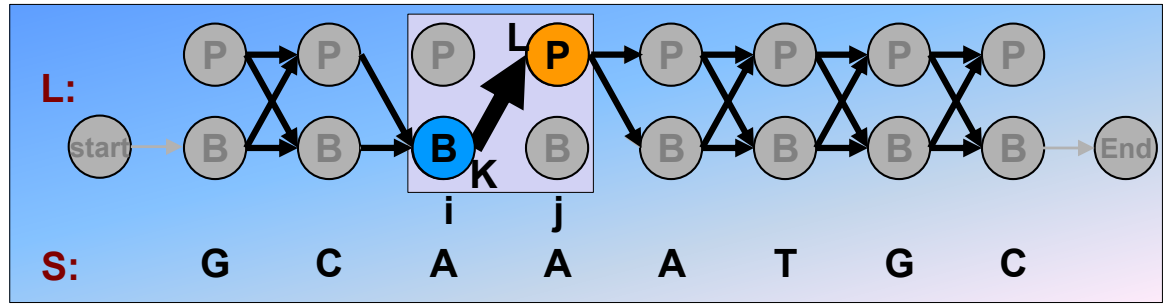
Principle: EXPECTATION MAXIMIZATION

1. Estimate probabilistic parse based on parameters (E step)
2. Update parameters A_{kl}, E_k based on probabilistic parse (M step)
3. Repeat 1 & 2, until convergence

Estimating probabilistic parse given params (E step)

To estimate $\mathbf{A_{kl}}$:

At each position i:



Find probability transition k→l is used:

$$P(\pi_i = k, \pi_{i+1} = l \mid x) = [1/P(x)] \times P(\pi_i = k, \pi_{i+1} = l, x_1 \ldots x_N) = Q/P(x)$$

where $Q = P(x_1 \ldots x_i, \pi_i = k, \pi_{i+1} = l, x_{i+1} \ldots x_N) =$

$= P(\pi_{i+1} = l, x_{i+1} \ldots x_N \mid \pi_i = k)\, P(x_1 \ldots x_i, \pi_i = k) =$

$= P(\pi_{i+1} = l, x_{i+1} x_{i+2} \ldots x_N \mid \pi_i = k)\, f_k(i) =$

$= P(x_{i+2} \ldots x_N \mid \pi_{i+1} = l)\, P(x_{i+1} \mid \pi_{i+1} = l)\, P(\pi_{i+1} = l \mid \pi_i = k)\, f_k(i) =$

$= b_l(i+1)\, e_l(x_{i+1})\, a_{kl}\, f_k(i)$

So:
$$\mathbf{P(\pi_i = k, \pi_{i+1} = l \mid x, \theta) = \frac{f_k(i)\, a_{kl}\, e_l(x_{i+1})\, b_l(i+1)}{P(x \mid \theta)}}$$

(For one such transition, at time step i→i+1)

New parameters given probabilistic parse (M step)

(Sum over all k→l transitions, at any time step i)

So,

$$A_{kl} = \sum_i P(\pi_i = k, \pi_{i+1} = l \mid x, \theta) = \sum_i \frac{f_k(i)\, a_{kl}\, e_l(x_{i+1})\, b_l(i+1)}{P(x \mid \theta)}$$

Similarly,

$$E_k(b) = [1/P(x)] \sum_{\{i \mid x_i = b\}} f_k(i)\, b_k(i)$$

Dealing with multiple training sequences

(Sum over all training seqs, all k→l transitions, all time steps i)

If we have several training sequences, $x^1, \ldots, x^M$, each of length N,

$$A_{kl} = \sum_x \sum_i P(\pi_i = k, \pi_{i+1} = l \mid x, \theta) = \sum_x \sum_i \frac{f_k(i)\, a_{kl}\, e_l(x_{i+1})\, b_l(i+1)}{P(x \mid \theta)}$$

Similarly,

$$E_k(b) = \sum_x (1/P(x)) \sum_{\{i \mid x^i = b\}} f_k(i)\, b_k(i)$$

The Baum-Welch Algorithm

Initialization:

Pick the best-guess for model parameters

(or arbitrary)

Iteration:

1. Forward
2. Backward
3. ➔ Calculate new log-likelihood $P(x \mid \theta)$ (E step)
4. Calculate A_{kl}, $E_k(b)$
5. ➔ Calculate new model parameters a_{kl}, $e_k(b)$ (M step)

GUARANTEED TO BE HIGHER BY EXPECTATION-MAXIMIZATION

Until $P(x \mid \theta)$ does not change much

The Baum-Welch Algorithm – comments

Time Complexity:

iterations $\times$ $O(K^2N)$

- Guaranteed to increase the log likelihood of the model

$P(\theta \mid x) = P(x, \theta) / P(x) = P(x \mid \theta) / (P(x) P(\theta))$

- Not guaranteed to find globally best parameters

Converges to local optimum, depending on initial conditions

- Too many parameters / too large model: Overtraining

	One path	All paths
Scoring	1. Scoring x, one path $P(x,\pi)$ Prob of a path, emissions	2. Scoring x, all paths $P(x) = \sum_{\pi} P(x,\pi)$ Prob of emissions, over all paths
Decoding	3. Viterbi decoding $\pi^* = \text{argmax}_{\pi} P(x,\pi)$ Most likely path	4. Posterior decoding $\pi\text{^} = \{\pi_i \mid \pi_i = \text{argmax}_k \sum_{\pi} P(\pi_i = k \mid x)\}$ Path containing the most likely state at any time point.
Learning	5. Supervised learning, given π $\Lambda^* = \text{argmax}_{\Lambda} P(x,\pi\mid\Lambda)$ 6. Unsupervised learning. $\Lambda^* = \text{argmax}_{\Lambda} \max_{\pi} P(x,\pi\mid\Lambda)$ Viterbi training, best path	6. Unsupervised learning $\Lambda^* = \text{argmax}_{\Lambda} \sum_{\pi} P(x,\pi\mid\Lambda)$ Baum-Welch training, over all paths

What have we learned ?

- Generative model. Hidden states, observed emissions.
  - Generate a random sequence
    - Choose random transition, choose random emission (#0)
- Scoring: Finding the likelihood of a given sequence
  - Calculate likelihood of annotated path and sequence
    - Multiply emission and transition probabilities (#1)
  - Without specifying a path, total probability of generating x
    - Sum probabilities over all paths
    - Forward algorithm (#3)
- Decoding: Finding the most likely path, given a sequence
  - What is the most likely path generating entire sequence?
    - Viterbi algorithm (#2)
  - What is the most probable state at each time step?
    - Forward + backward algorithms, posterior decoding (#4)
- Learning: Estimating HMM parameters from training data
  - When state sequence is known
    - Simply compute maximum likelihood A and E (#5a)
  - When state sequence is not known
    - Viterbi training: Iterative estimation of best path / frequencies (#5b)
    - Baum-Welch: Iterative estimation over all paths / frequencies (#6)